



RISC-V AI 扩展指令集

Tensor 扩展指令集

RACE 委员会 · 北京开源芯片研究院 · 北京算能科技有限公司联合发布

Beijing institute of open source chip

2026 年 · 北京

目录

1 目的.....	4
2 术语和定义.....	4
3 Tensor 扩展指令	5
3.1 CSR 寄存器.....	5
3.1.1 填充寄存器(Padding)	5
3.1.2 插入寄存器(Inserts).....	5
3.1.3 Stencil 寄存器(Stencil)	6
3.1.4 控制寄存器 0(Ctrl)	6
3.1.5 DMA Index 控制寄存器(Dma_Idx)	7
3.1.6 Sync Tag 控制寄存器(Sync_Tag).....	7
3.1.7 控制寄存器 1(Ctrl_1).....	7
3.1.8 其他控制寄存器	7
3.2 Tensor 配置寄存器.....	8
3.2.1 常量寄存器(CR).....	8
3.2.2 Tensor 寄存器(TR).....	9
3.2.3 Global Tensor 寄存器(GR)	10
3.3 Tensor 扩展指令格式	11
3.4 CSR 配置指令	12
3.5 TCR 配置指令.....	15
3.6 计算指令	16
3.6.1 卷积指令	16
3.6.2 矩阵乘指令.....	24
3.6.3 高性能矩阵乘指令	26
3.6.4 比较选择指令.....	30
3.6.5 Pooling 指令.....	31
3.6.6 整数算术指令.....	36
3.6.7 整数乘累加指令	38
3.6.8 整数比较指令.....	39
3.6.9 位运算指令.....	41
3.6.10 移位指令.....	43
3.6.11 浮点算术指令.....	45
3.6.12 浮点乘累加指令.....	47
3.6.13 浮点比较指令.....	48
3.6.14 数据精度类型转换指令.....	50
3.6.15 复制指令.....	53
3.6.16 浮点融合计算指令.....	54
3.6.17 整数量化与反量化指令.....	56
3.6.18 SFU 指令	58
3.6.19 Gather&Scatter 指令.....	61

3.6.20 CW 转置指令	65
3.6.21 SMEM 数据加载指令	66
3.6.22 LANE 复制指令	67
3.6.23 消息同步指令	68
3.7 DMA 指令	69
3.7.1 Tensor 数据加载指令	69
3.7.2 Tensor 数据存储指令	71
3.7.3 数据复制指令	72
3.7.4 转置指令	74
3.7.5 倒置指令	75
3.7.6 Mask&Index 指令	76
3.7.7 Gather&Scatter 指令	79
3.7.8 消息同步指令	82
3.8 同步指令	83

1 目的

本文档编写目的是为了详细描述 Tensor 扩展指令集架构以及相关功能设计，以便工具链开发、建模仿真等人员能够充分了解和利用该指令集进行开发工作。

2 术语和定义

序号	术语	定义说明
1.	DMA	Direct Memory Access, 用于全局存储与本地存储之间的数据搬运
2.	EU	Executing Unit, 执行单元
3.	LMEM	TPU 计算核心所属的 Level-1 SRAM
4.	L2M	所有 TPU 共享的 Level-2 SRAM
5.	GMEM	能访问的 DRAM 存储空间
6.	LANE_NUM	计算通道的数量
7.	EU_NUM	LANE 中 EU 的数量

3 Tensor 扩展指令

本章主要详细介绍 Tensor 扩展指令。Tensor 扩展指令按照 RISC-V 指令编码格式，使用的 major opcode(bit 6-0)皆为为 1011011。

3.1 CSR 寄存器

CSR 寄存器用于暂存指令部分属性信息。由于扩展指令的比特位不足，指令的很多其他信息须通过配置指令提前写入到 CSR 中，指令译码时硬件会读取 CSR 中的寄存器信息。CSR 寄存器如下：

3.1.1 填充寄存器(Padding)

用于配置卷积/Pooling 等运算对输入的 Feature Map 的上下左右做填充的参数信息：

Name	Bits	Offset	默认值	说明
pad_val	32	0	0	填充的常数值
up_pad	4	32	0	Feature Map 上边缘填充的行数
dn_pad	4	36	0	Feature Map 下边缘填充的行数
lf_pad	4	40	0	Feature Map 左边缘填充的列数
rt_pad	4	44	0	Feature Map 右边缘填充的列数
pad_mode	4	48	0	填充的模式： 0: Constant, 填入 pad_val 指定的常数； 1: Reflection; 2: Replication; 3: Circular
reserved	12	52	0	未定义

3.1.2 插入寄存器(Inserts)

用于配置卷积/Pooling 等运算对输入的 Feature Map 或 Kernel 进行元素间进行插值的参数信息：

Name	Bits	Offset	默认值	说明
insert_val	32	0	0	插入的常数值
insert_i_w	4	32	0	对 Feature Map 横向元素间插值的个数
insert_i_h	4	36	0	对 Feature Map 纵向元素间插值的个数
insert_k_w	4	40	0	对 Kernel 横向元素间插值的个数
insert_k_h	4	44	0	对 Kernel 纵向元素间插值的个数
reserved	16	48	0	未定义

3.1.3 Stencil 寄存器(Stencil)

用于配置卷积/Pooling 等运算的滑动窗口尺寸、移动步长等的参数信息：

Name	Bits	Offset	默认值	说明
kw	16	0	0	滑动窗口的宽度，单位为元素个数
kh	16	16	0	滑动窗口的高度，单位为元素个数
stride_w	4	32	0	窗口横向滑动步长，单位为元素个数
stride_h	4	36	0	窗口纵向滑动步长，单位为元素个数
reserved	14	40	0	未定义
krotate	1	62	0	是否对 Kernel 进行旋转 0：不旋转； 1：旋转
do_relu	1	63	0	是否对计算的结果做 Relu 操作 0：不做 1：做 Relu

3.1.4 控制寄存器 0(Ctrl)

用于控制指令行为的寄存器，包含控制 Rounding 模式、饱和模式以及 SFU 指令迭代次数等信息：

Name	Bits	Offset	默认值	说明
round_mode	4	0	0	浮点或整数右移 Rounding 模式 0: Round to even 1: Half round up 2: Round to zero 3: Floor 4: Ceiling 5: round to Plus Infinity 6: round to Minus Infinity
saturate	1	12	0	卷积/矩阵乘/池化及算术指令计算时是否支持对结果进行饱和 0: 不饱和 1: 饱和，浮点结果为 $\pm\text{Inf}$ 时饱和到最大或最小的 Normal 值，整型结果时饱和 MAX_VAL 或 MIN_VAL
sym_saturate	1	13	0	在整型卷积/矩阵乘/池化/算术指令计算时是否对结果做对称饱和 0: 不做对称饱和 1: 对称饱和，饱和到 $\pm((1 \ll (\text{bitwidth}-1))-1)$
rsqrt_iter	4	20	0	控制 Rsqrt 指令做牛顿迭代的次数
quant_reg_id	5	24	0	软件不可见

pwr_step	4	32	0	控制计算指令计算时的 power 的级别，分为 0~15 个级别，15 级为最大
kzp_reg_id	5	56	0	软件不可见

3.1.5 DMA Index 控制寄存器(Dma_Idx)

用于控制 GDMA_Gather/Scatter 指令行为的寄存器：

Name	Bits	Offset	默认值	说明
default_const_val	32	0	0	当 Gather/Scatter 的 Index 操作范围时填入的默认值
start_pos	32	32	0	当 Gather/Scatter 时的 Index 的偏移值

3.1.6 Sync Tag 控制寄存器(Sync_Tag)

用于读取同步指令状态的寄存器：

Name	Bits	Offset	默认值	说明
sync_tag	32	0	0	当 sync.i 指令执行完后，写入 tag 到该寄存器中，CPU 可以检查检查寄存器中的值是否为之前所设的 tag；读取该寄存器后会自动清 0

3.1.7 控制寄存器 1(Ctrl_1)

用于控制指令行为的寄存器：

Name	Bits	Offset	默认值	说明
tf32	1	4	0	控制 CONV/MM2 指令是否进入 TF32 计算模式
jump_cnt	16	32	0	控制 PRNG 设置 seed 后生成第 1 笔随机数需要跳过的轮数

3.1.8 其他控制寄存器

用于控制指令行为的寄存器：

Name	Bits	Offset	默认值	说明
en_breakp	1	0	0	写 1 开启 dgb_mode，后续的计算或 DMA 指令进入 debug 模式；写 0 则关闭
dis_auto_dep_id	1	1	0	控制是否关闭自动根据指令的 lmem 地址自动计算 dep_id 的机制，当值为 1 时表示需要软件通过指令 parallel 配置指令控制 dep_id 的生成

3.2 Tensor 配置寄存器

共有 40 个 Tensor 配置寄存器（TCR）作为指令的操作数，TCR 存储了 Tensor 的 layout、shape、stride、数据类型等信息，但未包含 Tensor 的数据。TCR 按照种类可分常数寄存器 CR（R0~R7），Tensor 寄存器 TR（R8~R31），Global Tensor 寄存器 GR（R32~R39）：

3.2.1 常量寄存器(CR)

共有 8 个常量寄存器（R0~R7），别名为 ca0~ca7，用于表示常数操作数，可以通过配置指令进行配置，如下所示。其中 ca0 是恒为 0 的 CR 寄存器，不能通过配置指令更改 ca0 的值：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
val[31:0]																															
teew				sub						val[55:32]																					

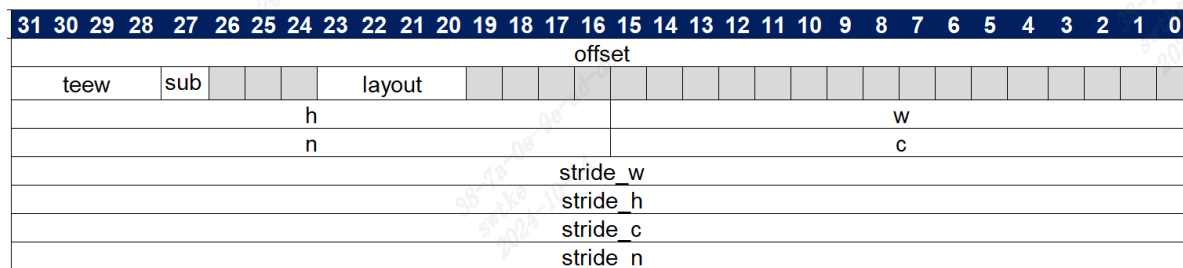
Name	Bits	Offset	默认值	说明
val	56	0	0	表示常数的值，最大支持 56 bit
reserved	3	56	0	未定义
subtype	1	59	0	用于区分整数的有符、无符或浮点的变种
teew	4	60	0	表示数据的位宽： 0: e8, 8 bit 1: e16, 16 bit 2: e32, 32 bit 5: e4, 4 bit

其中，subtype 可配合 teew 以及指令类型用于表示操作数具体数据类型：

指令类型	subtype	teew	所表示操作数的数据类型
整型指令	0	e4	U4
	1	e4	S4
	0	e8	U8
	1	e8	S8
	0	e16	U16
	1	e16	S16
	0	e32	U32
	1	e32	S32
浮点指令	0	e8	F8E5
	1	e8	F8E4
	0	e16	FP16
	1	e16	BF16
	0	e32	FP32

3.2.2 Tensor 寄存器(TR)

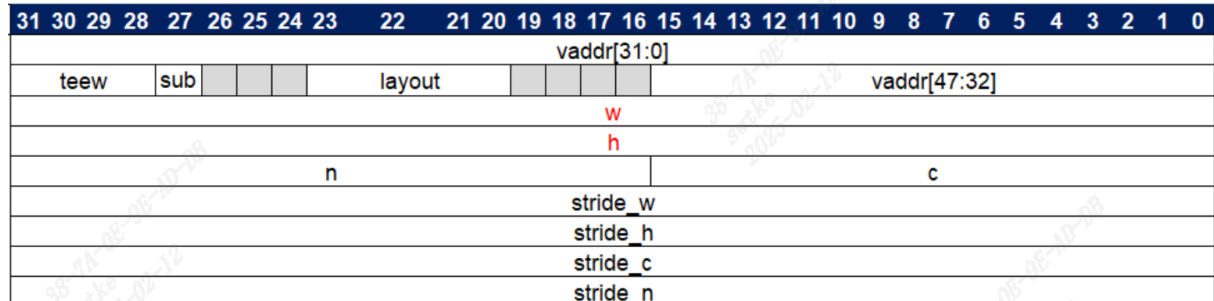
共有 24 个 Tensor 寄存器（R8~R31），别名为 ta0~ta7，用于表示存储在 LMEM 上的 Tensor，可以通过配置指令进行配置，如下所示：



Name	Bits	Offset	默认值	说明
offset	32	0	0	表示 Tensor 在 LMEM 上的相对起始地址
reserved	20	32	0	未定义
layout	4	52	0	表示 Tensor 在 LMEM 上的摆放模式 0: HW_ALIGN_MODE, hw 对齐排列模式) 1: CONTINOUS_MODE, 紧密排列模式 2: ROW_ALIGN_MODE, 行对齐排列模式 3: FREE_MODE, 自定义 Strides 模式
reserved	3	56	0	未定义
subtype	1	59	0	用于区分整数的有符无符或浮点的变种
teew	4	60	0	表示数据的位宽: 0: e8, 8 bit 1: e16, 16 bit 2: e32, 32 bit 6: e4, 4 bit
w	16	64	0	表示 Tensor W 维的长度
h	16	80	0	表示 Tensor H 维的长度
c	16	96	0	表示 Tensor C 维的长度
n	16	112	0	表示 Tensor N 维的长度
stride_w	32	128	0	表示 Tensor W 维的 Stride
stride_h	32	160	0	表示 Tensor H 维的 Stride
stride_c	32	192	0	表示 Tensor C 维的 Stride
stride_n	32	224	0	表示 Tensor N 维的 Stride

3.2.3 Global Tensor 寄存器(GR)

共有 8 个 Global Tensor 寄存器（R32~R39），别名为 ga0~ga7，用于表示存储在 GMEM 上的 Tensor，可以通过配置指令进行配置，如下所示：



Name	Bits	Offset	默认值	说明
vaddr	48	0	0	表示 Tensor 在 GMEM/SMEM 上的地址
reserved	4	48	0	未定义
layout	4	52	0	表示 Tensor 在 LMEM 上的摆放模式 0: HW_ALIGN_MODE, hw 对齐排列模式) 1: CONTINOUS_MODE, 紧密排列模式 2: ROW_ALIGN_MODE, 行对齐排列模式 3: FREE_MODE, 自定义 Strides 模式
reserved	3	56	0	未定义
subtype	1	59	0	用于区分整数的有符无符或浮点的变种
teew	4	60	0	表示数据的位宽： 0: e8, 8 bit 1: e16, 16 bit 2: e32, 32 bit 6: e4, 4 bit
w	32	64	0	表示 Tensor W 维的长度
h	32	96	0	表示 Tensor H 维的长度
c	16	128	0	表示 Tensor C 维的长度
n	16	144	0	表示 Tensor N 维的长度
stride_w	32	160	0	表示 Tensor W 维的 Stride
stride_h	32	192	0	表示 Tensor H 维的 Stride
stride_c	32	224	0	表示 Tensor C 维的 Stride
stride_n	32	256	0	表示 Tensor N 维的 Stride

3.3 Tensor 扩展指令格式

Tensor 扩展指令包含 32bit 与 64bit 的指令格式，具体格式分类如下所示：

(1) CSR 配置指令格式：

sf. vc. x																				
31-28	27	26	25	24	23	22	21	20	19-15					14-12	11	10	9	8	7	6-0
0000	1	0	1	func5					rs					100	imm5					1011011

(2) TCR 配置指令格式：

sf. vc. x																				
31-28	27	26	25	24	23	22	21	20	19-15					14-12	11	10	9	8	7	6-0
0000	1	1	1	func5					rs					100	ta/ga					1011011

(3) 计算指令格式：

sf. vc. i																							
31-28	27	26	25	24	23	22	21	20	19	18	17	16	15	14-12	11	10	9	8	7	6-0			
0000	0	func2		ts4					ts3						011	ts2					1011011		
0000	1	0	1	0	0	0	func2h		td						011	ts1					1011011		

sf. vc. i																				
31-28	27	26	25	24	23	22	21	20	19	18	17	16	15	14-12	11	10	9	8	7	6-0
0000	0	func5				imm2			ts3						011	ts2				1011011
0000	1	0	1	func5h				td						011	ts1				1011011	

(4) DMA 指令格式：

sf. vc. i																				
31-28	27	26	25	24	23	22	21	20	19	18	17	16	15	14-12	11	10	9	8	7	6-0
0000	0	func5				src2						src1	011	src1				101101		
0000	1	1	1	func4			dst						011	imm5				101101		

3.4 CSR 配置指令

通过 CSR 配置指令，可以将整数通用寄存器 **rs** 中的值写入到对应的 CSR 寄存器中，具体指令如下：

<code>rvt_cfg_satu</code>	<code>rs</code>
<code>rvt_cfg_pad</code>	<code>rs</code>
<code>rvt_cfg_insrt</code>	<code>rs</code>
<code>rvt_cfg_stencil</code>	<code>rs</code>
<code>rvt_cfg_round_mode</code>	<code>rs</code>
<code>rvt_cfg_rsqrtr_iter</code>	<code>rs</code>
<code>rvt_cfg_dmaidx</code>	<code>rs</code>
<code>rvt_cfg_quant</code>	<code>ts</code>

3.4.1.1 rvt_cfg_satu

<code>rvt_cfg_satu</code>	<code>rs</code>
---------------------------	-----------------

饱和配置指令。指令分别将 **rs** 的值写入 **CSR.Ctrl.saturate**；并且将 **rs[1:1]** 值写入到 **CSR.Ctrl.sym_saturate** 中。指令的计算逻辑如下所示：

<code>CSR.ctrl.saturate = rs[0:0];</code>

其中，**CSR.Ctrl.saturate** 用于控制带饱和量化功能的指令的饱和类型，如 **rvt_conv**、**rvt_mm**、**rvt_mm2** 等指令，如果 **CSR.Ctrl.saturate** 为 1，将对符号整型数做对称饱和；**CSR.ctrl.saturate** 用于控制浮点指令在做精度 **narrowing** 时是否饱和到最大/最小的 **normal** 数。

3.4.1.2 rvt_cfg_pad

<code>rvt_cfg_pad</code>	<code>rs</code>
--------------------------	-----------------

填充配置指令。指令将 **rs** 的值写入到 **CSR.Padding** 寄存器中。指令的计算逻辑如下所示：

<code>CSR.padding.pad_val = rs[31:0];</code>
<code>CSR.padding.up_pad = rs[35:32];</code>
<code>CSR.padding.dn_pad = rs[39:36];</code>
<code>CSR.padding.lf_pad = rs[43:40];</code>
<code>CSR.padding.rt_pad = rs[47:44];</code>
<code>CSR.padding.pad_mode = rs[51:48];</code>

该指令主要用于控制卷积类、**pooling** 类指令对输入的 **Feature Map** 的做四周的填充行为。

3.4.1.3 rvt_cfg_insrt

<code>rvt_cfg_insrt</code>	<code>rs</code>
----------------------------	-----------------

插值配置指令。指令将 `rs` 的值写入到 `CSR.Inserts` 寄存器中。指令的计算逻辑如下所示：

<pre>CSR.inserts.insert_val = rs[31:0]; CSR.inserts.insert_i_w = rs[35:32]; CSR.inserts.insert_i_h = rs[39:36]; CSR.inserts.insert_k_w = rs[43:40]; CSR.inserts.insert_k_h = rs[47:44];</pre>

该指令主要用于控制卷积类、`pooling` 类指令对输入的 Feature Map 或 Kernel 权重做插值的行为。

3.4.1.4 rvt_cfg_stencil

<code>rvt_cfg_stencil</code>	<code>rs</code>
------------------------------	-----------------

Stencil 模式配置指令。指令将 `rs` 的值写入到 `CSR.Stencil` 寄存器中。指令的计算逻辑如下所示：

<pre>CSR.stencil.kw = rs[15:0]; CSR.stencil.kh = rs[31:16]; CSR.stencil.stride_w = rs[39:32]; CSR.stencil.stride_h = rs[47:40]; CSR.stencil.krotate = rs[62:62]; CSR.stencil.do_relu = rs[63:63];</pre>

该指令主要用于控制卷积类、`pooling` 类指令的滑动窗口、步长、是否做 Kernel 的旋转以及是否对结果做 `relu` 等。

3.4.1.5 rvt_cfg_round_mode

<code>rvt_cfg_round_mode</code>	<code>rs</code>
---------------------------------	-----------------

Rounding 模式配置指令。指令将 `rs` 的值写入到 `CSR.Ctrl.round_mode` 寄存器中。指令的计算逻辑如下所示：

<pre>CSR.ctrl.round_mode = rs[3:0];</pre>

该指令主要用于控制计算指令浮点精度转换或整数右移时的取整模式。

3.4.1.6 rvt_cfg_rsqrt_iter

<code>rvt_cfg_rsqrt_iter</code>	<code>rs</code>
---------------------------------	-----------------

平方根倒数的牛顿迭代次数配置指令。指令将 `rs` 的值写入到 `CSR.Ctrl.rsqrt_iter` 寄存器中。指令的计算逻辑如下所示：

<pre>CSR.ctrl.rsqrt_iter = rs[3:0];</pre>

该指令主要用于控制 `rvt_rsqrt`、`rvt_fdiv` 指令的牛顿迭代次数。

3.4.1.7 `rvt_cfg_dmaidx`

<code>rvt_cfg_dmaidx</code>	<code>rs</code>
-----------------------------	-----------------

DMA Index 参数配置指令。指令将 `rs` 的值写入到 `CSR.Dma_Idx` 寄存器中。指令的计算逻辑如下所示：

<pre>CSR.dma_idx.default_const_val = rs[31:0]; CSR.dma_idx.start_pos = rs[63:32];</pre>

该指令主要用于 `rvt_hscatter`、`rvt_hgather`、`rvt_nzidx` 等指令配置 Index 初始值或默认写回值等参数。

3.4.1.8 `rvt_cfg_quant`

<code>rvt_cfg_quant</code>	<code>ts</code>
----------------------------	-----------------

后量化操作数控制指令。指令将 `ts` 寄存器的 ID 写入到 `CSR.Ctrl.quant_reg_id` 寄存器中。指令的计算逻辑如下所示：

<pre>CSR.ctrl.quant_reg_id = ts.id;</pre>

该指令主要用于指定计算指令的后量化操作数，该操作数可以是 TR 寄存器，代表量化参数为 Tensor，做 per-channel 量化；也可以是 CR 寄存器，代表量化参数为常数，做 per-tensor 量化。

3.4.1.9 `rvt_cfg_kzp`

<code>rvt_cfg_kzp</code>	<code>ts</code>
--------------------------	-----------------

权重偏置操作数控制指令。指令将 `ts` 寄存器的 ID 写入到 `CSR.Ctrl.kzp_reg_id` 寄存器中。指令的计算逻辑如下所示：

<pre>CSR.ctrl.kzp_reg_id = ts.id;</pre>

该指令主要用于指定 `rvt_conv` 或 `rvt_mm2` 等指令的权重偏置操作数，该操作数可以是 TR 寄存器，代表偏置是 Tensor；也可以是 CR 寄存器，代表偏置是常数。

3.5 TCR 配置指令

通过 TCR 配置指令，可以将整数通用寄存器 `rs` 中的值写入到对应的 TCR 寄存器中，用于配置 TCR 的属性。具体指令如下：

<code>rvt_cr</code>	<code>ca,rs</code>	# 将 <code>rs</code> 的值写入到 CR 寄存器[63:0]域中
<code>rvt_tr</code>	<code>ta,rs</code>	# 将 <code>rs</code> 的值写入到 TR 寄存器[63:0]域中
<code>rvt_gr</code>	<code>ga,rs</code>	# 将 <code>rs</code> 的值写入到 GR 寄存器[63:0]域中

3.5.1.1 rvt_cr

<code>rvt_cr</code>	<code>ca,rs</code>
---------------------	--------------------

CR 寄存器属性配置指令。指令将 `rs` 的值写入 CR 寄存器的前 64 bit 中。指令的计算逻辑如下所示：

<pre>ca.val = rs[55:0]; ca.subtype = rs[59:59]; ca.teew = rs[63:60];</pre>
--

该指令主要用于指定 CR 寄存器所代表常数的数据类型，以及对应的常数值。

3.5.1.2 rvt_tr

<code>rvt_tr</code>	<code>ta,rs</code>
---------------------	--------------------

TR 寄存器属性配置指令。指令将 `rs` 的值写入 TR 寄存器的前 64 bit 中。指令的计算逻辑如下所示：

<pre>ta.offset = rs[31:0]; ta.layout = rs[55:52]; ta.subtype = rs[59:59]; ta.teew = rs[63:60];</pre>
--

该指令主要用于指定 TR 寄存器所代表的 Tensor 的数据类型，以及 Tensor 在 LMEM 上的相对地址。

3.5.1.3 rvt_gr

<code>rvt_gr</code>	<code>ga,rs</code>
---------------------	--------------------

GR 寄存器属性配置指令。指令将 `rs` 的值写入 GR 寄存器的前 64 bit 中。指令的计算逻辑如下所示：

<pre>ga.vaddr = rs[47:0]; ta.layout = rs[55:52]; ga.subtype = rs[59:59]; ga.teew = rs[63:60];</pre>

该指令主要用于指定 GR 寄存器所代表 Tensor 的数据类型，以及 Tensor 在 GMEM 或 SMEM 上的地址。

3.6 计算指令

3.6.1 卷积指令

```
rvt_conv    out,x,w,bias,_rq
rvt_conva   out,x,w,bias
rvt_fconv   out,x,w,bias,_rq
rvt_fconva   out,x,w,bias
rvt_dwconv   out,x,w,bias,_rq
rvt_fdwconv out,x,w,bias,_rq
```

3.6.1.1 rvt_conv

```
rvt_conv    out,x,w,bias,_rq
```

整型卷积指令。指令对输入的张量 Feature Map (x)与 Kernel 权重(w)做 2D 卷积，并且支持：

- 对 Feature Map 做边缘 Padding，由 CSR.Padding 控制；
- 对 Feature Map 和 Kernel 做元素插值，由 CSR.Inserts 控制；
- 对 Kernel 做旋转，由 CSR.Stencil.krotate 控制；
- 对 Kernel 减去偏置值 kzp，由 CSR.Ctrl.kzp_reg_id 控制；
- 调整卷积窗口的尺寸和滑动步长，由 CSR.Stencil 控制；
- 累加偏置 Bias；
- 对结果做 Relu 操作，由 CSR.Stencil.do_relu 控制；
- 支持对结果做饱和和量化，量化到不同的精度，由 _rq 和 CSR.quant_reg_id 控制；
- 支持调整量化时 rounding 方式，由 CSR.Ctrl.round_mode 控制；

指令的计算逻辑如下所示：

```
x = inserts(x, CSR.inserts);
x = padding(x, CSR.padding);
if (CSR.stencil.krotate)
    w = kernel_rotate(w);
w -= kzp;
w = kernel_inserts(w, CSR.inserts);
tmp = conv2d(x, w) + bias;
if (CSR.stencil.do_relu)
    tmp = max(tmp, 0);
if (_rq)
    tmp = requant(tmp, quant, CSR.ctrl.round_mode);
out = satu(tmp, out.teew, out.subtype);
```

指令各操作数的约束如下所示：

(1) 当_rq 为 1 时:

	Opd	Dtype	Shape	Layout	Note
源 操作数	x	S8/U8	(n, ic, ih, iw)	HW 对齐排列或 Free mode	输入 Feature Map
	w	S8/U8	(1, oc, 1, div_up(ic,64)*kh*kw*64)	紧密排列或常数	输入 Kernel 权重
	bias	S32/U32	(1, oc, 1, 1)	紧密排列或常数	偏置
	kzp	S16	(1, oc, 1, 1)	紧密排列或常数	Kernel 偏置
	quant	U32	(1, oc, 1, 2)	紧密排列或常数	量化参数 Tensor (1,c,1,0[31:0]) 为 multiplier; (1,c,1,1[7:0]) 为 shift; (1,c,1,1[31:16]) 为 offset; 当为常数时: CR.val[31:0]为 multiplier; CR.val[39:32]为 shift; CR.val[55:40]为 offset;
目的 操作数	out	S8/U8/ S16/U16	(n, oc, oh, ow)	HW 对齐排列	输出 Tensor

(2) 当_rq 为 0 时:

	Opd	Dtype	shape	layout	Note
源 操作数	x	S8/U8	(n, ic, ih, iw)	HW 对齐排列或 Free mode	输入 Feature Map
	w	S8/U8	(1, oc, 1, div_up(ic,64)*kh*kw*64)	紧密排列或常数	输入 Kernel 权重
	bias	S32/U32	(1, oc, 1, 1)	紧密排列或常数	偏置
目的 操作数	out	S32/U32	(n, oc, oh, ow)	HW 对齐排列	输出 Tensor

3.6.1.2 rvt_conva

```
rvt_conva out,x,w,bias
```

整型卷积累加指令。指令对输入的张量 Feature Map (x)与 Kernel 权重(w) 做 2D 卷积，再与前一次结果累加并输出到张量 out 中。且指令还支持：

- 对 Feature Map 做边缘 Padding，由 CSR.Padding 控制；
- 对 Feature Map 和 Kernel 做元素插值，由 CSR.Inserts 控制；
- 对 Kernel 做旋转，由 CSR.Stencil.krotate 控制；
- 对 Kernel 减去偏置 kzp，由 CSR.Ctrl.kzp_reg_id 控制；
- 调整卷积窗口的尺寸和滑动步长，由 CSR.Stencil 控制；
- 累加偏置 Bias；
- 对结果做 Relu 操作，由 CSR.Stencil.do_relu 控制；

指令的计算逻辑如下所示：

```

x = inserts(x, CSR.inserts);
x = padding(x, CSR.padding);
if (CSR.stencil.kroate)
    w = kernel_rotate(w);
w -= kzp;
w = kernel_inserts(w, CSR.inserts);
out += conv2d(x, w)
out += bias;
if (CSR.stencil.do_relu)
    out = max(out, 0);

```

指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout
源操作数	x	S8/U8	(n, ic, ih, iw)	HW 对齐排列或 Free mode
	w	S8/U8	(1, oc, 1, div_up(ic,64)*kh*kw*64)	紧密排列或常数
	bias	S32/U32	(1, oc, 1, 1)	紧密排列或常数
	kzp	S16	(1, oc, 1, 1)	紧密排列或常数
目的操作数	out	S32/U32	(n, oc, oh, ow)	HW 对齐排列

3.6.1.3 rvt_fconv

```
rvt_fconv out, x, w, bias, _rq
```

浮点卷积指令。指令对输入的张量 Feature Map (x)与 Kernel 权重(w) 做 2D 卷积，并输出结果到张量 out 中。且指令还支持：

- 对 Feature Map 做边缘 Padding，由 CSR.Padding 控制；
- 对 Feature Map 和 Kernel 做元素插值，由 CSR.Inserts 控制；
- 对 Kernel 做旋转，由 CSR.Stencil.krotate 控制；
- 调整卷积窗口的尺寸和滑动步长，由 CSR.Stencil 控制；
- 累加偏置 Bias；
- 对结果做 Relu 操作，由 CSR.Stencil.do_relu 控制；
- 支持 FP8 卷积时对结果做饱和量化，量化到不同的精度，由_rq 和 CSR.quant_reg_id 控制；
- 支持结果精度转换时的饱和方式，由 CSR.Ctrl.saturate 控制；

指令的计算逻辑如下所示：

```

x = inserts(x, CSR.inserts);
x = padding(x, CSR.padding);
if (CSR.stencil.kroate)
    w = kernel_rotate(w);
w = kernel_inserts(w, CSR.inserts);
tmp = conv2d(x, w) + bias;
if (CSR.stencil.do_relu)
    tmp = max(tmp, 0);
if (_rq)
    tmp = tmp*quant;
out = FloatToFloat(tmp, out.teew, out.subtype, RTN,
                    CSR.ctrl.saturate);

```

指令各操作数的约束如下所示：

	Opd	Dtype					shape	layout
源操作数	x	F8E4/ F8E5	BF16	FP16	FP32	FP32 (TF32)	(n, ic, ih, iw)	HW 对齐排列或 Free mode
	w	F8E4/ F8E5	BF16	FP16	FP32	FP32 (TF32)	FP8: (1, oc, 1, div_up(ic,64)*kh*kw*64) FP16/BF16: (1, oc, 1, div_up(ic,32)*kh*kw*32) TF32: (1, oc, 1, div_up(ic,16)*kh*kw*16) FP32: (1, oc, 1, ic*kh*kw)	紧密排列或 常数
	bias	FP16/ FP32	BF16/ FP32	FP16/ FP32	FP32	FP32	(1, oc, 1, 1)	紧密排列或 常数
	quant	FP32	--	--	--	--	(1, oc, 1, 1)	紧密排列或 常数
目的操作数	out	F8E4/ F8E5/ BF16/ FP16/ FP32	BF16/ FP32	FP16/ FP32	FP32	FP32	(n, oc, oh, ow)	HW 对齐排列

3.6.1.4 rvt_fconva

```
rvt_fconva out, x, w, bias
```

浮点卷积累加指令。指令对输入的张量 Feature Map (x) 与 Kernel 权重(w) 做 2D 卷积，并累加前一次结果后输出到张量 out 中。且指令还支持：

- 对 Feature Map 做边缘 Padding，由 CSR.Padding 控制；
- 对 Feature Map 和 Kernel 做元素插值，由 CSR.Inserts 控制；
- 对 Kernel 做旋转，由 CSR.Stencil.krotate 控制；
- 调整卷积窗口的尺寸和滑动步长，由 CSR.Stencil 控制；
- 累加偏置 Bias；

➤ 对结果做 Relu 操作，由 CSR.Stencil.do_relu 控制；

指令的计算逻辑如下所示：

```
x = inserts(x, CSR.inserts);
x = padding(x, CSR.padding);
if (CSR.stencil.krotate)
    w = kernel_rotate(w);
w = kernel_inserts(w, CSR.inserts);
tmp = conv2d(x, w)
tmp += out;
tmp += bias;
if (CSR.stencil.do_relu)
    tmp = max(tmp, 0);
out = FloatToFloat(tmp, out.teew, out.subtype, RTN,
                    CSR.ctrl.saturate);
```

指令各操作数的约束如下所示：

	Opd	Dtype					shape	layout
源操作数	x	F8E4/ F8E5	BF16	FP16	FP32	FP32 (TF32)	(n, ic, ih, iw)	HW 对齐排列或 Free mode
	w	F8E4/ F8E5	BF16	FP16	FP32	FP32 (TF32)	FP8: (1, oc, 1, div_up(ic,64)*kh*kw*64) FP16/BF16: (1, oc, 1, div_up(ic,32)*kh*kw*32) TF32: (1, oc, 1, div_up(ic,16)*kh*kw*16) FP32: (1, oc, 1, ic*kh*kw)	紧密排列或常数
	bias	FP16/ FP32	BF16/ FP32	FP16/ FP32	FP32	FP32	(1, oc, 1, 1)	紧密排列或常数
目的操作数	out	FP32					(n, oc, oh, ow)	HW 对齐排列

3.6.1.5 rvt_dwconv

```
rvt_dwconv out,x,w,bias,_rq
```

整型深度卷积指令。指令对输入的张量 Feature Map (x) 与 Kernel 权重(w) 做 2D 卷积，并且支持：

- 对 Feature Map 做边缘 Padding，由 CSR.Padding 控制；
- 对 Feature Map 和 Kernel 做元素插值，由 CSR.Inserts 控制；
- 对 Kernel 做旋转，由 CSR.Stencil.krotate 控制；
- 调整卷积窗口的尺寸和滑动步长，由 CSR.Stencil 控制；
- 累加偏置 Bias；
- 对结果做 Relu 操作，由 CSR.Stencil.do_relu 控制；

- 支持对结果做量化，量化到不同的精度，由_rq 和 CSR.quant_reg_id 控制；
- 支持调整量化时 rounding 方式，由 CSR.Ctrl.round_mode 控制；
- 支持对结果做饱和；

指令的计算逻辑如下所示：

```
x = inserts(x, CSR.inserts);
x = padding(x, CSR.padding);
if (CSR.stencil.krotate)
    w = kernel_rotate(w);
w = kernel_inserts(w, CSR.inserts);
tmp = conv2d(x, w) + bias;
if (CSR.stencil.do_relu)
    tmp = max(tmp, 0);
if (_rq)
    tmp = requant(tmp, quant, CSR.ctrl.round_mode);
out = satu(tmp, out.teew, out.subtype);
```

指令各操作数的约束如下所示：

(1) 当_rq 为 1 时：

	Opd	Dtype	shape	layout
源 操作数	x	S8/U8	(n, c, ih, iw)	HW 对齐排列
	w	S8/U8	(1, c, kh, kw)	紧密排列或常数
	bias	S32	(1, c, 1, 1)	紧密排列或常数
	quant	U32	(1, c, 1, 2)	紧密排列或常数
目的 操作数	out	S8/U8/ S16/U16/ S32/U32	(n, c, oh, ow)	HW 对齐排列

(2) 当_rq 为 0 时：

	Opd	Dtype	shape	layout
源 操作数	x	S8/U8	(n, c, ih, iw)	HW 对齐排列或 Free mode
	w	S8/U8	(1, c, kh, kw)	紧密排列或常数
	bias	S32	(1, oc, 1, 1)	紧密排列或常数
目的 操作数	out	S32	(n, oc, oh, ow)	HW 对齐排列

3.6.1.6 rvt_fdwconv

```
rvt_fdwconv out, x, w, bias, _rq
```

浮点深度卷积指令。指令对输入的张量 Feature Map (x) 与 Kernel 权重(w) 做 2D 卷积，并且支持：

- 对 Feature Map 做边缘 Padding，由 CSR.Padding 控制；
- 对 Feature Map 和 Kernel 做元素插值，由 CSR.Inserts 控制；

- 对 Kernel 做旋转，由 CSR.Stencil.krotate 控制；
- 调整卷积窗口的尺寸和滑动步长，由 CSR.Stencil 控制；
- 累加偏置 Bias；
- 对结果做 Relu 操作，由 CSR.Stencil.do_relu 控制；
- 支持 FP8 深度卷积时对结果做量化，量化到不同的精度，由 _rq 和 CSR.quant_reg_id 控制；
- 支持浮点精度转换时是否饱和，由 saturate 控制；

指令的计算逻辑如下所示：

```
x = inserts(x, CSR.inserts);
x = padding(x, CSR.padding);
if (CSR.stencil.krotate)
    w = kernel_rotate(w);
w = kernel_inserts(w, CSR.inserts);
tmp = conv2d(x, w) + bias;
if (CSR.stencil.do_relu)
    tmp = max(tmp, 0);
if (_rq)
    tmp *= quant;
out = FloatToFloat(tmp, out.teew, out.subtype, RTN,
                    CSR.ctrl.saturate);
```

指令各操作数的约束如下所示：

(1) 当_rq 为 1 时：

	Opd	Dtype				shape	layout
源操作数	x	F8E4/ F8E5	BF16	FP16	FP32	(n, c, ih, iw)	HW 对齐排列或 Free mode
	w	F8E4/ F8E5	BF16	FP16	FP32	(1, c, kh, kw)	紧密排列或常数
	bias	FP16	BF16	FP16	FP32	(1, c, 1, 1)	紧密排列或常数
	quant	FP32	--	--	--	(1, c, 1, 1)	紧密排列或常数
目的操作数	out	F8E4/ F8E5/ FP16/ BF16/ FP32	BF16	FP16	FP32	(n, c, oh, ow)	HW 对齐排列

(2) 当_rq 为 0 时：

	Opd	Dtype				shape	layout
源操作数	x	F8E4/ F8E5	BF16	FP16	FP32	(n, c, ih, iw)	HW 对齐排列或 Free mode
	w	F8E4/ F8E5	BF16	FP16	FP32	(1, c, kh, kw)	紧密排列或常数
	bias	FP16	BF16	FP16	FP32	(1, c, 1, 1)	紧密排列或常数
目的操作数	out	FP32	FP32	FP32	FP32	(n, c, oh, ow)	HW 对齐排列

3.6.1.7 rvt_fconvdw

```
rvt_fconvdw out, x, dy
```

浮点深度卷积指令。指令对输入的张量 Feature Map (x) 与输出梯度权重(dy) 做 2D 卷积得到权重的梯度(dw)，并且支持：

- 对 Feature Map 做边缘 Padding，由 CSR.Padding 控制；
- 对 Feature Map 和 Kernel 做元素插值，由 CSR.Inserts 控制；
- 调整卷积窗口的尺寸和滑动步长，由 CSR.Stencil 控制；

指令的计算逻辑如下所示：

```
x = inserts(x, CSR.inserts);  
x = padding(x, CSR.padding);  
dy = kernel_inserts(dy, CSR.inserts);  
psum = conv2d(x, dy);  
out = FloatToFloat(psum, out.teew, out.subtype, RTN, 0);
```

指令各操作数的约束如下所示：

	Opd	Dtype				shape	layout
源操作数	x	F8E4/ F8E5	BF16	FP16	FP32	(n, ic, ih, iw)	HW 对齐排列或 Free mode
	dy	F8E4/ F8E5	BF16	FP16	FP32	(n, oc, oh, ow)	对齐排列
目的操作数	out	FP32	FP32	FP32	FP32	FP16/BF16/FP8 时: (1,oc,div_up(ic,32)*kh*kw,32) TF32 时: (1,oc,div_up(ic,16)*kh*kw,16) FP32 : (1,oc,ic*kh*kw,1)	紧密排列

3.6.2 矩阵乘指令

```
rvt_fmm.<nn|tn>    out,x,w,bias,_relu  
rvt_fmml.<nn|tn>    out,x,w,bias,_relu
```

3.6.2.1 rvt_fmm_nn,rvt_fmm_tn

```
rvt_fmm_nn    out,x,w,bias,_relu  
rvt_fmm_tn    out,x,w,bias,_relu
```

浮点矩阵乘指令。指令对输入矩阵 X 和 W 做矩阵乘计算，并将结果写入到矩阵 out 中。且指令支持：

- 累加偏置 Bias；
- 对结果做 Relu 操作，由 _relu 控制；

指令的计算逻辑如下所示：

```
out = matmul(x, w) + bias;  
if (_relu)  
    out = max(out, 0)
```

指令各操作数的约束如下所示：

	Opd	Dtype	Shape	Layout
源 操作数	x	FP32	rvt_mm_nn: (m, div_up(k,w ₀), 1, w ₀) rvt_mm_tn: (k, div_up(m,w ₀), 1, w ₀)	HW 对齐排列或 常数
	w	FP32	(k, div_up(n,w ₁), 1, w ₁)	紧密排列
	bias	FP32	(1, div_up(n,w ₁), 1, w ₁)	紧密排列或常数
目的 操作数	out	FP32	(m, div_up(n,w ₁), 1, w ₁)	HW 对齐排列

3.6.2.2 rvt_fmml_nn,rvt_fmml_tn

```
rvt_fmml_nn    out,x,w,bias,_relu  
rvt_fmml_tn    out,x,w,bias,_relu
```

浮点矩阵乘累加指令。指令对输入矩阵 X 和 W 做矩阵乘计算，并累加前一次结果后输出到张量 out 中。且指令还支持：

- 累加偏置 Bias；
- 对结果做 Relu 操作，由 CSR.Stencil.do_relu 控制；

指令的计算逻辑如下所示：


```

out += matmul(x, w);
out += bias;
if (_relu)
    out = max(out, 0);

```

指令各操作数的约束如下所示：

	Opd	Dtype	Shape	Layout
源 操作数	x	FP32	rvt_mm_nn: (m, div_up(k,w ₀), 1, w ₀) rvt_mm_tn: (k, div_up(m,w ₀), 1, w ₀)	HW 对齐排列或 常数
	w	FP32	(k, div_up(n,w ₁), 1, w ₁)	紧密排列
	bias	FP32	(1, div_up(n,w ₁), 1, w ₁)	紧密排列或常数
目的 操作数	out	FP32	(m, div_up(n,w ₁), 1, w ₁)	HW 对齐排列

3.6.3 高性能矩阵乘指令

```
rvt_mm2.<nn|nt|tt> out,x,w,bias,_rq,_relu
rvt_mm2a.<nn|nt|tt> out,x,w,bias,_relu
rvt_fmm2.<nn|nt|tt> out,x,w,bias,_rq,_relu
rvt_fmm2a.<nn|nt|tt> out,x,w,bias,_relu
```

3.6.3.1 rvt_mm2_nn, rvt_mm2_nt, rvt_mm2_tt

```
rvt_mm2_nn out,x,w,bias,_rq,_relu
rvt_mm2_nt out,x,w,bias,_rq,_relu
rvt_mm2_tt out,x,w,bias,_rq,_relu
```

整型矩阵乘指令,可以实现相比 rvt_mm 指令多倍的加速。指令对输入矩阵 X 和 W 做矩阵乘计算,并将结果写入到矩阵 out 中。且指令支持:

- 累加偏置 Bias;
- 对结果做 Relu 操作,由 _relu 控制;
- 支持对结果做量化,量化到不同的精度,由 _rq 和 CSR.quant_reg_id 控制;
- 支持调整量化的 rounding 以及饱和方式,由 CSR.Ctrl.round_mode 控制;

指令的计算逻辑如下所示:

```
tmp = matmul(x, (w-kzp)) + bias;
if (_relu)
    tmp = max(tmp, 0);
if (_rq)
    tmp = requant(tmp, quant, CSR.ctrl.round_mode);
out = satu(tmp, out.teew, out.subtype);
```

指令各操作数的约束如下所示:

(1) 当 _rq 为 1 时:

	Opd	Dtype	shape			layout	Note
			rvt_mm2_nn	rvt_mm2_nt	rvt_mm2_tt		
源操作数	x	S8/U8	(1, m, 1, k)	(1, m, 1, k)	(1, k, 1, m)	HW 对齐排列或常数	左矩阵
	w	S8/U8	(1, k, 1, n)	(1, n, 1, k)	(1, n, 1, k)	HW 对齐排列	右矩阵
	bias	S32/U32	(1, lane_num, 1, n)	(1, lane_num, 1, n)	(1, n, 1, 1)	nn/nt: HW 对齐排列或常数 tt: 紧密排列或常数	偏置
	kzp	S16	(1, lane_num, 1, n)	(1, lane_num, 1, n)	(1, n, 1, 1)	nn/nt: HW 对齐排列或常数 tt: 紧密排列或常数	右矩阵偏置
	quant	U32	(1, lane_num, n, 2)	(1, lane_num, n, 2)	(1, n, 1, 2)	nn/nt: HW 对齐排列或	量化 Tensor (1,c,1,0[31:0]) 为 multiplier;

						常数 tt: 紧密排列或常数	(1,c,1,1[7:0]) 为 shift; (1,c,1,1[31:16]) 为 offset; 当为常数时: cr.val[31:0]为 multiplier; cr.val[39:32]为 shift; cr.val[55:40]为 offset;
目的 操作数	out	S8/U8/ S16/U16	(1, m, 1, n)	(1, m, 1, n)	(1, n, 1, m)	HW 对齐排列	

(2) 当_rq 为 0 时:

	Opd	Dtype	shape			layout
			rvt_mm2_nn	rvt_mm2_nt	rvt_mm2_tt	
源 操作数	x	S8/U8	(1, m, 1, k)	(1, m,1,k)	(1, k,1,m)	HW 对齐排列 或常数
	w	S8/U8	(1, k, 1, n)	(1, n, 1, k)	(1, n, 1, k)	HW 对齐排列
	bias	S32/U32	(1, lane_num,1,n)	(1,lane_num,1,n)	(1, n, 1, 1)	nn/nt: HW 对 齐排列或常数 tt: 紧密排列 或常数
	kzp	S16	(1, lane_num,1,n)	(1,lane_num,1,n)	(1,n,1,1)	nn/nt: HW 对 齐排列或常数 tt: 紧密排列 或常数
	quant	--	--	--	--	--
目的 操作数	out	S32/U32	(1, m, 1, n)	(1, m, 1, n)	(1, n, 1, m)	HW 对齐排列

3.6.3.2 rvt_mm2a_nn, rvt_mm2a_nt, rvt_mm2a_tt

```

rvt_mm2a_nn  out,x,w,bias,_relu
rvt_mm2a_nt  out,x,w,bias,_relu
rvt_mm2a_tt  out,x,w,bias,_relu

```

整型矩阵乘累加指令, 可以实现相比 rvt_mma 指令多倍的加速。指令对输入矩阵 X 和 W 做矩阵乘计算，再与前一次结果累加并输出到张量 out 中。且指令还支持：

- 累加偏置 Bias;
- 对结果做 Relu 操作，由_relu 控制;

指令的计算逻辑如下所示：

```

out += matmul(x, (w-kzp))
out += bias;
if (_relu)
    out = out(tmp,0);

```

指令各操作数的约束如下所示：

	Opd	Dtype	shape			layout
			rvt_mm2_nn	rvt_mm2_nt	rvt_mm2_t t	

源 操作数	x	S8/U8	(1, m, 1, k)	(1, m, 1, k)	(1, k, 1, m)	HW 对齐排列 或常数
	w	S8/U8	(1, k, 1, n)	(1, n, 1, k)	(1, n, 1, k)	HW 对齐排列
	bias	S32/U32	(1, lane_num, 1, n)	(1, lane_num, 1, n)	(1, n, 1, 1)	紧密排列或常数
	kzp	S16	(1, lane_num, 1, n)	(1, lane_num, 1, n)	(1, n, 1, 1)	紧密排列或常数
目的 操作数	out	S32/U32	(1, m, 1, n)	(1, m, 1, n)	(1, n, 1, m)	HW 对齐排列

3.6.3.3 rvt_fmm2_nn, rvt_fmm2_nt, rvt_fmm2_tt

```
rvt_fmm2_nn out, x, w, bias, _rq, _relu
rvt_fmm2_nt out, x, w, bias, _rq, _relu
rvt_fmm2_tt out, x, w, bias, _rq, _relu
```

浮点矩阵乘指令, 可以实现相比 rvt_fmm 指令多倍的加速。指令对输入矩阵 X 和 W 做矩阵乘计算, 并将结果写入到矩阵 out 中。且指令支持:

- 累加偏置 Bias;
- 对结果做 Relu 操作, 由 _relu 控制;
- 支持浮点精度转换时是否饱和, 由 CSR.Ctrl.saturate 控制;

指令的计算逻辑如下所示:

```
tmp = matmul(x, w) + bias;
if (_relu)
    tmp = max(tmp, 0)
if (_rq)
    tmp *= quant;
out = FloatToFloat(tmp, out.teew, out.subtype, RTN,
    CSR.ctrl.saturate);
```

指令各操作数的约束如下所示:

	Opd	Dtype			shape			layout
					rvt_mm2_nn	rvt_mm2_nt	rvt_mm2_tt	
源 操作数	x	F8E4/ F8E5	FP16	BF16	(1, m, 1, k)	(1, m, 1, k)	(1, k, 1, m)	HW 对齐排列 或常数
	w	F8E4/ F8E5	FP16	BF16	(1, k, 1, n)	(1, n, 1, k)	(1, n, 1, k)	HW 对齐排列
	bias	FP16/ FP32	FP16/ FP32	FP32	(1, lane_num, 1, n)	(1, lane_num, 1, n)	(1, n, 1, 1)	紧密排列或常数
	quant	FP32	--	--	(1, lane_num, 1, n)	(1, lane_num, 1, n)	(1, n, 1, 1)	紧密排列或常数
目的 操作数	out	F8E4/ F8E5/ FP16/ BF16/	FP16/ FP32	BF16/ FP32	(1, m, 1, n)	(1, m, 1, n)	(1, n, 1, m)	HW 对齐排列

		FP32						
--	--	------	--	--	--	--	--	--

3.6.3.4 rvt_fmm2a_nn,rvt_fmm2a_nt,rvt_fmm2a_tt

```
rvt_fmm2a_nn out,x,w,bias,_rq,_relu
rvt_fmm2a_nt out,x,w,bias,_rq,_relu
rvt_fmm2a_tt out,x,w,bias,_rq,_relu
```

浮点矩阵乘累加指令,可以实现相比 rvt_mm 指令多倍的加速。指令对输入矩阵 X 和 W 做矩阵乘计算，并累加前一次结果后输出到张量 out 中。且指令还支持：

- 累加偏置 Bias;
- 对结果做 Relu 操作，由 CSR.Stencil.do_relu 控制;

指令的计算逻辑如下所示：

```
out += matmul(x, w);
out += bias;
if (_relu)
out = max(out,0);
if (_rq)
tmp *= quant;
out = FloatToFloat(tmp, out.teew,out.subtype, RTN,
CSR.ctrl.saturate);
```

指令各操作数的约束如下所示：

	Opd	Dtype			shape			layout
					rvt_mm2_nn	rvt_mm2_nt	rvt_mm2_tt	
源 操作数	x	F8E5/ F8E4	FP16	BF16	(1, m, 1, k)	(1, m,1,k)	(1, k,1,m)	HW 对齐排列 或常数
	w	F8E5/ F8E4	FP16	BF16	(1, k, 1, n)	(1, n, 1, k)	(1, n, 1, k)	HW 对齐排列
	bias	FP16/ FP32	FP16/ FP32	FP16/ FP32	(1, lane_num,1,n)	(1,lane_num,1,n)	(1, n, 1, 1)	紧密排列或常 数
	quant	FP32	--	--	(1, lane_num, 1,n)	(1,lane_num, 1,n)	(1, n, 1, 1)	紧密排列或常 数
目的 操作数	out	F8E4/ F8E5/ FP16/ BF16/ FP32	FP16/ FP32	BF16/ FP32	(1, m, 1, n)	(1, m, 1, n)	(1, n, 1, m)	HW 对齐排列

3.6.4 比较选择指令

```
rvt_cmplt out,a,b,c,d      # out = a<b?c:d
rvt_cmpeq out,a,b,c,d      # out = a==b?c:d
rvt_cmpgt out,a,b,c,d      # out = a>b?a:b
rvt_fcmplt out,a,b,c,d     # out = a<b?c:d
rvt_fcmpeq out,a,b,c,d     # out = a==b?c:d
rvt_fcmpgt out,a,b,c,d     # out = a>b?c:d
```

3.6.4.1 rvt_cmplt, rvt_cmpeq, rvt_cmpgt

```
rvt_cmplt out,a,b,c,d      # out = a<b?c:d
rvt_cmpeq out,a,b,c,d      # out = a==b?c:d
rvt_cmpgt out,a,b,c,d      # out = a>b?a:b
```

整型比较选择指令。指令对 Tensor A 和 B 进行逐数据比较，并根据结果选择对应 Tensor C 和 D 中的数据。指令各操作数的约束如下所示：

	Opd	Dtype	Shape	Layout
源 操作数	a	S8/U8/S16/U16/ S32/U32	(n, c, h, w)	HW 对齐排列 或 Stride(0,0,0,0) 或常数
	b	同 a	(n, c, h, w)	HW 对齐排列 或 Stride(0,0,0,0) 或常数
	c	W8/W16/W32	(n, c, h, w)	HW 对齐排列 或常数
	d	同 c	(n, c, h, w)	HW 对齐排列 或常数
目的 操作数	out	同 c	(n, c, h, w)	HW 对齐排列

3.6.4.2 rvt_fcmplt, rvt_fcmpeq, rvt_fcmpgt

```
rvt_fcmplt out,a,b,c,d     # out = a<b?c:d
rvt_fcmpeq out,a,b,c,d     # out = a==b?c:d
rvt_fcmpgt out,a,b,c,d     # out = a>b?c:d
```

浮点比较选择指令。指令对 Tensor A 和 B 进行逐数据比较，并根据结果选择对应 Tensor C 和 D 中的数据。指令各操作数的约束如下所示：

	Opd	Dtype	Shape	Layout
源 操作数	a	F8E4/F8E5/ FP16/BF16/FP32	(n, c, h, w)	HW 对齐排列 或 Stride(0,0,0,0) 或常数
	b	同 a	(n, c, h, w)	HW 对齐排列 或 Stride(0,0,0,0) 或常数
	c	W8/W16/W32	(n, c, h, w)	HW 对齐排列 或常数
	d	同 c	(n, c, h, w)	HW 对齐排列 或常数

目的 操作数	out	同 c	(n, c, h, w)	HW 对齐排列
-----------	-----	-----	--------------	---------

3.6.5 Pooling 指令

```

rvt_pool_avg    out, x, w, _rq
rvt_pool_max    out, x
rvt_pool_min    out, x
rvt_pool_favg   out, x, w
rvt_pool_fmax   out, x
rvt_pool_fmin   out, x

```

3.6.5.1 rvt_pool_avg

```

rvt_pool_avg    out, x, w, _rq

```

整型平均池化指令。指令对输入的张量 Feature Map 做 2D 平均池化计算，并且支持：

- 对 Feature Map 做边缘 Padding，由 CSR.Padding 控制；
- 对 Feature Map 元素插值，由 CSR.Inserts 控制；
- 调整池化窗口的尺寸和滑动步长，由 CSR.Stencil 控制；
- 对结果做 Relu 操作，由 CSR.Stencil.do_relu 控制；
- 支持对结果做量化，量化到不同的精度，由 _rq 和 CSR.quant_reg_id 控制；
- 支持调整量化时 rounding 方式，由 CSR.Ctrl.round_mode 控制；
- 支持对结果做饱和；

指令的计算逻辑如下所示：

```

x = padding(x, CSR.padding);
x = inserts(x, CSR.inserts);
for n,c,oh,ow in out.shape:
    sum = avg_pooling(x(n,c,:, :), CSR.stencil)
    out(n,c,oh,ow) = sum * w;
if (CSR.stencil.do_relu)
    out = max(out,0);
if (_rq)
    out = requant(out,quant,CSR.ctrl.round_mode);
out = satu(out,out.teew,out.subtype);

```

指令各操作数的约束如下所示：

(3) 当_rq 为 1 时：

	Opd	Dtype	Shape	layout	Note
源 操作数	x	S8/U8	(n, c, ih, iw)	HW 对齐排列	输入 Feature Map
	w	S8/U8	--	常数	窗口权重参数
	quant	U32	(1, c, 1, 2)	紧密排列 或常数	量化参数 Tensor (1,c,1,0[31:0]) 为 multiplier; (1,c,1,1[7:0]) 为 shift; (1,c,1,1[31:16]) 为 offset; 当为常数时：

					cr.val[31:0]为 multiplier; cr.val[39:32]为 shift; cr.val[55:40]为 offset;
目的 操作数	out	S8/U8/ S16/U16	(n, c, oh, ow)	HW 对齐排列	输出 Tensor

(4) 当_rq 为 0 时:

	Opd	Dtype	Shape	Layout	Note
源 操作数	x	S8/U8	(n, c, ih, iw)	HW 对齐排列	输入 Feature Map
	w	S8/U8	--	常数	窗口权重参数
	quant	--	--	--	--
目的 操作数	out	S32	(n, c, oh, ow)	HW 对齐排列	输出 Tensor

3.6.5.2 rvt_pool_max

```
rvt_pool_max out, x
```

整型最大池化指令。指令对输入的张量 Feature Map 做 2D 最大池化计算，并且支持：

- 对 Feature Map 做边缘 Padding，由 CSR.Padding 控制；
- 对 Feature Map 元素插值，由 CSR.Inserts 控制；
- 调整池化窗口的尺寸和滑动步长，由 CSR.Stencil 控制；
- 对结果做 Relu 操作，由 CSR.Stencil.do_relu 控制；

指令的计算逻辑如下所示：

```
x = padding(x, CSR.padding);
x = inserts(x, CSR.inserts);
for (n, c, oh, ow in out.shape)
    out(n, c, oh, ow) = max_pooling(x(n, c, :, :), CSR.stencil);
if (CSR.stencil.do_relu)
    out = max(out, 0);
```

指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout
源操作数	x	S8/U8	(n, c, ih, iw)	HW 对齐排列
目的操作数	out	同 x	(n, c, oh, ow)	HW 对齐排列

3.6.5.3 rvt_pool_min

```
rvt_pool_min out, x
```

整型最小池化指令。指令对输入的张量 Feature Map 做 2D 最小池化计算，并且支持：

- 对 Feature Map 做边缘 Padding，由 CSR.Padding 控制；
- 对 Feature Map 元素插值，由 CSR.Inserts 控制；

- 调整池化窗口的尺寸和滑动步长，由 CSR.Stencil 控制；
- 对结果做 Relu 操作，由 CSR.Stencil.do_relu 控制；

指令的计算逻辑如下所示：

```
x = padding(x, CSR.padding);
x = inserts(x, CSR.inserts);
out = 0;
for (n,c,oh,ow in out.shape)
    out(n,c,oh,ow) = max_pooling(x(n,c,:,:), CSR.stencil);
if (CSR.stencil.do_relu)
    out = max(out,0);
```

指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout
源操作数	x	S8/U8	(n, ic, ih, iw)	HW 对齐排列
目的操作数	out	同 x	(n, oc, oh, ow)	HW 对齐排列

3.6.5.4 rvt_pool_favg

```
rvt_pool_favg out,x,w,_rq
```

浮点平均池化指令。指令对输入的张量 Feature Map 做 2D 平均池化计算，并且支持：

- 对 Feature Map 做边缘 Padding，由 CSR.Padding 控制；
- 对 Feature Map 元素插值，由 CSR.Inserts 控制；
- 调整池化窗口的尺寸和滑动步长，由 CSR.Stencil 控制；
- 对结果做 Relu 操作，由 CSR.Stencil.do_relu 控制；
- 支持 FP8 深度卷积时对结果做量化，量化到不同的精度，由 _rq 和 CSR.quant_reg_id 控制；
- 支持浮点精度转换时是否饱和，由 saturate 控制；

指令的计算逻辑如下所示：

```
x = padding(x, CSR.padding);
x = inserts(x, CSR.inserts);
out = 0;
for n,c,oh,ow in out.shape:
    for px in window(x(n,c,:,:),oh,ow, CSR.stencil):
        out(n,c,oh,ow) += px * w;
if (CSR.stencil.do_relu)
    out = max(out,0);
if (_rq)
    out *= quant;
```

```
out = FloatToFloat(tmp, out.teew, out.subtype, RTN,
                    CSR.ctrl.saturate);
```

指令各操作数的约束如下所示：

	Opd		Dtype	Shape	layout
源操作数	x	F8E5/F8E4	FP16/BF16/FP32	(n, c, ih, iw)	HW 对齐排列
	w	F8E5/F8E4	同 x	--	常数
	quant	FP32	FP32	(1, c, 1, 1)	紧密排列或常数
目的操作数	out	FP8/FP16/BF16/FP32	同 x	(n, c, oh, ow)	HW 对齐排列

3.6.5.5 rvt_pool_fmax

```
rvt_pool_fmax out, x
```

浮点最大池化指令。指令对输入的张量 Feature Map 做 2D 最大池化计算，并且支持：

- 对 Feature Map 做边缘 Padding，由 CSR.Padding 控制；
- 对 Feature Map 元素插值，由 CSR.Inserts 控制；
- 调整池化窗口的尺寸和滑动步长，由 CSR.Stencil 控制；
- 对结果做 Relu 操作，由 CSR.Stencil.do_relu 控制；

指令的计算逻辑如下所示：

```
x = padding(x, CSR.padding);
x = inserts(x, CSR.inserts);
out = 0;
for (n, c, oh, ow in out.shape)
    out(n, c, oh, ow) = max_pooling(x(n, c, :, :), CSR.stencil);
if (CSR.stencil.do_relu)
    out = max(out, 0);
```

指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout
源操作数	x	FP8/FP16/BF16/FP32	(n, c, ih, iw)	HW 对齐排列
目的操作数	out	同 x	(n, c, oh, ow)	HW 对齐排列

3.6.5.6 rvt_pool_fmin

```
rvt_pool_fmin out, x
```

浮点最小池化指令。指令对输入的张量 Feature Map 做 2D 最小池化计算，并且支持：

- 对 Feature Map 做边缘 Padding，由 CSR.Padding 控制；
- 对 Feature Map 元素插值，由 CSR.Inserts 控制；
- 调整池化窗口的尺寸和滑动步长，由 CSR.Stencil 控制；
- 对结果做 Relu 操作，由 CSR.Stencil.do_relu 控制；

指令的计算逻辑如下所示：

```
x = padding(x, CSR.padding);
x = inserts(x, CSR.inserts);
out = 0;
for (n,c,oh,ow in out.shape)
    out(n,c,oh,ow) = max_pooling(x(n,c,:,:), CSR.stencil);
if (CSR.stencil.do_relu)
    out = max(out,0);
```

指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout
源操作数	x	FP8/FP16/BF16/FP32	(n, c, ih, iw)	HW 对齐排列
目的操作数	out	同 x	(n, c, oh, ow)	HW 对齐排列

3.6.6 整数算术指令

```
rvt_add    out, a, b, shift
rvt_sub    out, a, b, shift
rvt_mul    out, a, b, shift
rvt_abs    out, a
```

3.6.6.1 rvt_add, rvt_sub, rvt_mul

```
rvt_add    out, a, b, shift
rvt_sub    out, a, b, shift
rvt_mul    out, a, b, shift
```

整型算术指令。指令对 Tensor A 和 Tensor B 做逐元素的加、减、乘计算。并且支持：

- 支持对结果做移位操作，移位位数由 Shift 控制，正数表示左移，负数表示右移，且支持右移的 Rounding 模式，由 CSR.Ctrl.round_mode 控制；
- 支持当操作数 a 或 b 的 n、h 或 w 维大小为 1 时，广播计算；
- 当结果位宽大于或者小于输入时，做符号扩充或截断操作；

指令的计算逻辑如下所示：

```
tmp = op(a, b);
tmp = ArithShiftRound(tmp, shift, CSR.ctrl.round_mode);
out = tmp;
```

指令各操作数的约束如下所示：

	Opd	Dtype	Shape	Layout
源操作数	a	INT8/INT16/INT32	(n, c, h, w) 或 (n/1, c, h/1, w/1)	HW 对齐排列/ 紧密排列/ Free Mode/ 常数
	b	INT8/INT16/INT32		
	shift	S8/S16/S32	(1, c, 1, 1)	紧密排列/常数
目的操作数	out	INT8/INT16/INT32	(n, c, h, w)	HW 对齐排列

指令各操作数的符号约束如下所示：

指令	a	b	shift	out
rvt_add rvt_mul	unsigned	unsigned	signed	unsigned
	signed	unsigned	signed	signed
	signed	signed	signed	signed
rvt_sub	unsigned/signed	unsigned/signed	signed	signed

指令各操作数的位宽约束如下所示：

指令	a	b	shift	out
rvt_add rvt_mul rvt_sub	INT8	INT8/INT16/INT32	S8	INT8/INT16/INT32
	INT16	INT8/INT16/INT32	S8/S16	INT8/INT16/INT32
	INT32	INT8/INT16/INT32	S8/S16/S32	INT8/INT16/INT32

3.6.6.2 rvt_abs

```
rvt_abs out, a
```

整型取绝对值指令。指令对 Tensor A 做取绝对值计算，并且支持：

- 支持当操作数 a 的 n、h 或 w 维大小为 1 时，广播计算。

指令的计算逻辑如下所示：

```
out = a<0?(-1*a):a;
```

指令各操作数的约束如下所示：

	Opd	Dtype			Shape	Layout
源操作数	a	S8/ U8	S16/ U16	S32/ U32	(n, c, h, w) 或 (n/1, c, h/1, w/1)	任意排列/常数
目的操作数	out	U8	U16	U32	(n, c, h, w)	任意排列

3.6.7 整数乘累加指令

```
rvt_mac out, a, b, shift
```

3.6.7.1 rvt_mac

```
rvt_mac out, a, b, shift
```

整型乘累加指令。对 Tensor A 和 B 进行乘加运算，并将结果写入到 Tensor Out 中去，并且支持：

- 支持累加 out 前，对 out 做左移，左移位数由 shift[15:8]控制；
- 支持对输出的结果做右移，右移位数由 shift[7:0]控制，右移的 Rounding 模式由 CSR.Ctrl.round_mode 控制；
- 支持对输出的结果做饱和；
- 支持当操作数 a 或 b 的 n、h 或 w 维大小为 1 时，广播计算。

指令的计算逻辑如下所示，注意：

- 当操作数 a 或 b 为有符型时，输出 out 只能为有符型。

```
subtype = a.subtype | b.subtype;
tmp = a * b + (out << shift[15:8]);
tmp = RightShiftRound(tmp, shift[7:0],
CSR.ctrl.round_mode);
out = satu(tmp, out.teew, subtype);
```

指令各操作数的约束如下所示：

	Opd	Dtype	Shape	Layout
源操作数	a	S8/U8	(n, c, h, w) 或 (n/1, c, h/1, w/1)	任意排列/ 常数
	b	S8/U8		
	shift	U16	--	常数 shift[15:8]为 out 初始值左移位数； shift[7:0]为结果输出前右边移位数；
目的操作数	out	S16/U16	(n, c, h, w)	任意排列

3.6.8 整数比较指令

```
rvt_max    out,a,b    # out = a>b?a:b
rvt_min    out,a,b    # out = a<b?a:b
rvt_sel_gt out,a,b,c   # out = a>b?c:0
rvt_sel_eq out,a,b,c   # out = a==b?c:0
rvt_sel_lt out,a,b,c   # out = a<b?c:0
```

3.6.8.1 rvt_max, rvt_min

```
rvt_max    out,a,b    # out = a>b?a:b
rvt_min    out,a,b    # out = a<b?a:b
```

整型比较指令。指令对 Tensor A 和 Tensor B 做逐元素做取大或取小值计算。并且支持：

- 支持当操作数 a 或 b 的 n、h 或 w 维大小为 1 时，广播计算。

指令的计算逻辑如下所示：

```
out = a>b?a:b; // rvt_max
out = a<b?a:b; // rvt_min
```

指令各操作数的约束如下所示：

	Opd	Dtype	Shape	Layout
源操作数	a	S8/U8/ S16/U16/S32/U32	(n, c, h, w) 或 (n/1, c, h/1, w/1)	任意排列/ 常数
	b	同 a		
目的操作数	out	同 a	(n, c, h, w)	任意排列

3.6.8.2 rvt_sel_gt, rvt_sel_eq, rvt_sel_lt

```
rvt_sel_gt out,a,b,c   # out = a>b?c:0
rvt_sel_eq out,a,b,c   # out = a==b?c:0
rvt_sel_lt out,a,b,c   # out = a<b?c:0
```

整型比较选择指令。指令对 Tensor A 和 Tensor B 做逐元素做比较，并根据比较的结果选择 Tensor C 的值或 0。并且支持：

- 支持当操作数 a 或 b 的 n、h 或 w 维大小为 1 时，广播计算。

指令的计算逻辑如下所示：

```
out = a>b?c:0; // rvt_sel_gt
out = a==b?a:b; // rvt_sel_eq
out = a<b?c:0; // rvt_sel_lt
```

指令各操作数的约束如下所示：

	Opd	Dtype	Shape	Layout
源操作数	a	S8/U8/ S16/U16/S32/U32	(n, c, h, w) 或 (n/1, c, h/1, w/1)	任意排列/ 常数
	b	同 a		
	c	同 a	常数	常数
目的操作数	out	同 a	(n, c, h, w)	任意排列

3.6.9 位运算指令

```
rvt_and out, a, b
rvt_xor out, a, b
rvt_or out, a, b
rvt_not out, a
```

3.6.9.1 rvt_and, rvt_xor, rvt_or

```
rvt_and out, a, b
rvt_xor out, a, b
rvt_or out, a, b
```

整型位操作指令。指令对 Tensor A 和 Tensor B 做逐元素做 AND、XOR、OR 计算。

并且支持：

- 支持当操作数 a 或 b 的 n、h 或 w 维大小为 1 时，广播计算。

指令的计算逻辑如下所示：

```
out = a&b; // rvt_and
out = a^b; // rvt_xor
out = a|b; // rvt_or
```

指令各操作数的约束如下所示：

	Opd	Dtype	Shape	Layout
源操作数	a	U8/U16/U32	(n, c, h, w) 或	任意排列/ 常数
	b	同 a	(n/1, c, h/1, w/1)	
目的操作数	out	同 a	(n, c, h, w)	任意排列

3.6.9.2 rvt_not

```
rvt_not out, a
```

整型取反指令。指令对 Tensor A 做取反计算。并且支持：

- 支持当操作数 a 的 n、h 或 w 维大小为 1 时，广播计算。

指令的计算逻辑如下所示：

```
out = ~a; // rvt_not
```

指令各操作数的约束如下所示：

	Opd	Dtype	Shape	Layout
源操作数	a	U8/U16/U32	(n, c, h, w) 或 (n/1, c, h/1, w/1)	任意排列/ 常数

目的操作数	out	同 a	(n, c, h, w)	任意排列
-------	-----	-----	--------------	------

3.6.10 移位指令

```
rvt_lshr  out,a,shift
rvt_ashr  out,a,shift
rvt_rshr  out,a,shift
```

3.6.10.1 rvt_lshr

```
rvt_lshr  out,a,shift
```

整型移位操作指令。指令对 Tensor A 做逐元素做逻辑移位计算。并且支持：

- 支持当操作数 a 或 shift 的 n、h 或 w 维大小为 1 时，广播计算。
- 支持当 shift 的值为正数时左移，为负数时右移；
- 支持右移时的 Rounding 模式，由 CSR.Ctrl.round_mode 控制

指令的计算逻辑如下所示：

```
out = shift>0?
      left_shift(a, shift):
      right_shift(a,abs(shift),CSR.ctrl.round_mode);
```

指令各操作数的约束如下所示，注意：

	Opd	Dtype	Shape	Layout
源操作数	a	S8/U8/S16/U16/S32/U32	(n, c, h, w) 或 (n/1, c, h/1, w/1)	任意排列/ 常数
	shift	S8		
目的操作数	out	同 a	(n, c, h, w)	任意排列

3.6.10.2 rvt_ashr

```
rvt_ashr  out,a,shift
```

整型移位操作指令。指令对 Tensor A 做逐元素做算术移位计算。并且支持：

- 支持当操作数 a 或 shift 的 n、h 或 w 维大小为 1 时，广播计算。
- 支持当 shift 的值为正数时左移，为负数时右移；
- 支持右移时的 Rounding 模式，由 CSR.Ctrl.round_mode 控制

指令的计算逻辑如下所示：

```
out = shift>0?
      left_shift(a, shift):
      arith_right_shift(a,abs(shift),CSR.ctrl.round_mode
);
```

指令各操作数的约束如下所示：

	Opd	Dtype	Shape	Layout
源操作数	a	S8/U8/S16/U16/S32/U32	(n, c, h, w) 或	任意排列/ 常数
	shift	S8	(n/1, c, h/1, w/1)	
目的操作数	out	同 a	(n, c, h, w)	任意排列

3.6.10.3 rvt_rshr

```
rvt_rshr out, a, shift
```

整型移位操作指令。指令对 Tensor A 做逐元素做绕回移位计算。并且支持：

- 支持当操作数 a 或 shift 的 n、h 或 w 维大小为 1 时，广播计算。
- 支持当 shift 的值为正数时左移，为负数时右移；

指令的计算逻辑如下所示：

```
out = shift>0?
    rotate_left_shift(a, shift):
    rotate_right_shift(a,abs(shift),CSR.ctrl.round_mod
e);
```

指令各操作数的约束如下所示：

	Opd	Dtype	Shape	Layout
源操作数	a	S8/U8/S16/U16/S32/U32	(n, c, h, w) 或	任意排列/ 常数
	shift	S8	(n/1, c, h/1, w/1)	
目的操作数	out	同 a	(n, c, h, w)	任意排列

3.6.11 浮点算术指令

对 Tensor A 和 B 进行加、减、乘、除法、取绝对值等算术运算，并将结果写入到 Tensor Out 中去。指令支持 FP8/FP16/BF16/FP32 等精度。具体指令如下：

```
rvt_fadd out,a,b
rvt_fsub out,a,b
rvt_fmul out,a,b
rvt_fdiv out,a,b
rvt_fabs out,a
```

3.6.11.1 rvt_fadd, rvt_fsub, rvt_fmul

```
rvt_fadd out,a,b
rvt_fsub out,a,b
rvt_fmul out,a,b
```

浮点算术指令。指令对 Tensor A 和 Tensor B 做逐元素的加、减、乘计算。并且支持：

- 支持是否对结果做饱和，由 CSR.Ctrl.saturate 控制；
- 支持当操作数 a 或 b 的 n、h 或 w 维大小为 1 时，广播计算。

指令的计算逻辑如下所示：

```
tmp = op(a,b);
if(CSR.ctrl.saturate)
    out = FloatToFloat(tmp, out.teew,out.subtype, RTN,
                       CSR.ctrl.saturate);
else
    out = tmp;
```

指令各操作数的约束如下所示：

	Opd	Dtype	Shape	Layout
源操作数	a	FP8/BF16/FP16/FP32	(n, c, h, w) 或 (n/1, c, h/1, w/1)	任意排列/ 常数
	b	同 a		
目的操作数	out	同 a	(n, c, h, w)	任意排列

3.6.11.2 rvt_fdiv

```
rvt_fdiv out,a,b
```

浮点除法算术指令。指令对 Tensor A 和 Tensor B 做逐元素的除法计算。并且支持：

- 支持是否对结果做饱和，由 CSR.Ctrl.saturate 控制；
- 支持调整倒数平方根迭代次数以改变计算结果精度，由 CSR.Ctrl.rsqrt_iter 控制；

- 支持当操作数 a 或 b 的 n、h 或 w 维大小为 1 时，广播计算。

指令的计算逻辑如下所示：

```
tmp = a * rsqrt(b, CSR.rsqrt_iter);
if(CSR.ctrl.saturate)
    tmp = FloatToFloat(tmp, out.teew,out.subtype, RTN,
                        CSR.ctrl.saturate);
out = tmp;
```

指令各操作数的约束如下所示：

	Opd	Dtype	Shape	Layout
源操作数	a	BF16/FP16/FP32	(n, c, h, w) 或 (n/1, c, h/1, w/1)	任意排列/ 常数
	b	同 a		
目的操作数	out	同 out	(n, c, h, w)	任意排列

3.6.11.3 rvt_fabs

```
rvt_fabs out,a
```

浮点取绝对值指令。指令对 Tensor A 做取绝对值计算，并且支持：

- 支持当操作数 a 的 n、h 或 w 维大小为 1 时，广播计算。

指令的计算逻辑如下所示：

```
out = a<0?(-1*a):a;
```

指令各操作数的约束如下所示：

	Opd	Dtype	Shape	Layout
源操作数	a	FP8/BF16/FP16/FP32	(n, c, h, w) 或 (n/1, c, h/1, w/1)	任意排列/ 常数
目的操作数	out	同 out	(n, c, h, w)	任意排列

3.6.12 浮点乘累加指令

```
rvt_fmac out,a,b
```

3.6.12.1 rvt_fmac

```
rvt_fmac out,a,b
```

浮点乘加指令。对 Tensor A 和 B 进行乘加运算，并将结果写入到 Tensor Out 中去，并且支持：

- 支持当操作数 a 或 b 的 n、h 或 w 维大小为 1 时，广播计算。
- 支持是否对结果做饱和，由 CSR.Ctrl.saturate 控制；

指令的计算逻辑如下所示：

```
tmp = out + a * b;  
if(CSR.ctrl.saturate)  
    tmp = FloatToFloat(tmp, out.teew,out.subtype, RTN,  
                        CSR.ctrl.saturate);  
out = tmp;
```

指令各操作数的约束如下所示：

	Opd	Dtype			Shape	Layout
源操作数	a	FP16	BF16	FP32	(n, c, h, w) 或 (n/1, c, h/1, w/1)	任意排列/ 常数
	b	FP16	BF16	FP32		
目的操作数	out	FP16	BF16	FP32	(n, c, h, w)	任意排列

3.6.13 浮点比较指令

```
rvt_fmax out, a, b
rvt_fmin out, a, b
rvt_fsel_gt out, a, b, c
rvt_fsel_eq out, a, b, c
rvt_fsel_lt out, a, b, c
```

3.6.13.1 rvt_fmax, rvt_fmin

```
rvt_fmax out, a, b
rvt_fmin out, a, b
```

浮点比较指令。指令对 Tensor A 和 Tensor B 做逐元素做取大或取小值计算。并且支持：

- 支持当操作数 a 或 b 的 n、h 或 w 维大小为 1 时，广播计算。

指令的计算逻辑如下所示：

```
out = a>b?a:b; // rvt_fmax
out = a<b?a:b; // rvt_fmin
```

指令各操作数的约束如下所示：

	Opd	Dtype	Shape	Layout
源操作数	a	FP8/FP16/BF16/FP32	(n, c, h, w) 或	任意排列/ 常数
	b	同 a	(n/1, c, h/1, w/1)	
目的操作数	out	同 a	(n, c, h, w)	任意排列

3.6.13.2 rvt_fsel_gt, rvt_fsel_eq, rvt_fsel_lt

```
rvt_fsel_gt out, a, b, c
rvt_fsel_eq out, a, b, c
rvt_fsel_lt out, a, b, c
```

浮点比较选择指令。指令对 Tensor A 和 Tensor B 做逐元素做比较，并根据比较的结果选择 Tensor C 的值或 0。并且支持：

- 支持当操作数 a 或 b 的 n、h 或 w 维大小为 1 时，广播计算。

指令的计算逻辑如下所示：

```
out = a>b?c:0; // rvt_fsel_gt
out = a==b?a:b; // rvt_fsel_eq
out = a<b?c:0; // rvt_fsel_lt
```

指令各操作数的约束如下所示：

	Opd	Dtype	Shape	Layout
源操作数	a	FP8/FP16/BF16/FP32	(n, c, h, w) 或	任意排列/ 常数
	b	同 a	(n/1, c, h/1, w/1)	
	c	同 a	常数	常数
目的操作数	out	同 a	(n, c, h, w)	任意排列

3.6.14 数据精度类型转换指令

对 Tensor A 进行数据精度类型转换操作，并将结果写入到 Tensor Out 中去。指令支持 INT4/INT8/INT16/INT32/FP16/BF16/FP32 等类型间的互转。具体指令如下：

```
rvt_cvt_i2i out,a  
rvt_cvt_i2f out,a  
rvt_cvt_f2i out,a  
rvt_cvt_f2f out,a
```

3.6.14.1 rvt_cvt_i2i

```
rvt_cvt_i2i out,a
```

整型精度转换指令。指令对 Tensor A 做整型精度转换，并且支持：

- 当做精度 narrow 转换时，支持饱和操作；
- 支持有符型和无符号型间的互转；
- 支持当操作数 a 的 n、h 或 w 维大小为 1 时，广播计算。

指令的计算逻辑如下所示：

```
if (bits_of(out.teew) > bits_of(a.teew))  
    out = i2i_cvt(x);  
else  
    out = i2i_cvt_satu(x);
```

指令各操作数的约束如下所示：

	Opd	Dtype	Shape	Layout
源操作数	a	S4/U4/S8/U8/ S16/U16/S32/U32	(n, c, h, w) 或 (n/1, c, h/1, w/1)	任意排列/ 常数
目的操作数	out	S4/U4/S8/U8/ S16/U16/S32/U32	(n, c, h, w)	任意排列

3.6.14.2 rvt_cvt_i2f

```
rvt_cvt_i2f out,a
```

整型到浮点精度转换指令。指令对 Tensor A 做整型到浮点精度转换，并且支持：

- 支持是否做饱和，由 CSR.Ctrl.saturate 控制；
- 支持尾数的舍入模式，由 CSR.Ctrl.round_mode 控制；
- 支持对浮点结果做饱和；
- 支持当操作数 a 的 n、h 或 w 维大小为 1 时，广播计算。

指令的计算逻辑如下所示：

```
out = i2f_cvt(x, CSR.ctrl.saturate);
```

指令各操作数的约束如下所示：

	Opd	Dtype	Shape	Layout
源操作数	a	S4/U4/S8/U8/ S16/U16/S32/U32	(n, c, h, w) 或 (n/1, c, h/1, w/1)	任意排列/ 常数
目的操作数	out	FP8/FP16/BF16/FP32	(n, c, h, w)	任意排列

3.6.14.3 rvt_cvt_f2i

```
rvt_cvt_f2i out, a
```

浮点到整型精度转换指令。指令对 Tensor A 做浮点到整型的精度转换，并且支持：

- 支持饱和操作；
- 支持取整时的 Rounding 模式，由 CSR.Ctrl.round_mode 控制；
- 支持当操作数 a 的 n、h 或 w 维大小为 1 时，广播计算。

指令的计算逻辑如下所示：

```
out = f2i_cvt(x, CSR.ctrl.round_mode);
```

指令各操作数的约束如下所示：

	Opd	Dtype	Shape	Layout
源操作数	a	FP8/FP16/BF16/FP32	(n, c, h, w) 或 (n/1, c, h/1, w/1)	任意排列/ 常数
目的操作数	out	S4/U4/S8/U8/S16/U16/S 32/U32	(n, c, h, w)	任意排列

3.6.14.4 rvt_cvt_f2f

```
rvt_cvt_f2f out, a
```

浮点精度转换指令。指令对 Tensor A 做浮点精度转换，并且支持：

- 支持是否做饱和操作，由 CSR.Ctrl.saturate 控制；
- 支持精度转换时的 Rounding 模式，由 CSR.Ctrl.round_mode 控制；
- 支持当操作数 a 的 n、h 或 w 维大小为 1 时，广播计算。

指令的计算逻辑如下所示：

```
out = f2f_cvt(x, CSR.ctrl.round_mode, CSR.ctrl.saturate);
```

指令各操作数的约束如下所示：

	Opd	Dtype	Shape	Layout
源操作数	a	FP8/FP16/BF16/FP32	(n, c, h, w) 或 (n/1, c, h/1, w/1)	任意排列/ 常数
目的操作数	out	FP8/FP16/BF16/FP32	(n, c, h, w)	任意排列

3.6.15 复制指令

```
rvt_cp      out,a
```

3.6.15.1 rvt_cp

```
rvt_cp      out,a
```

张量复制指令。并且支持：

- 支持当操作数 a 的 n、h 或 w 维大小为 1 时，做对应维度的广播复制。

指令的计算逻辑如下所示：

```
out = f2f_cvt(x, CSR.ctrl.round_mode, CSR.ctrl.saturate);
```

指令各操作数的约束如下所示：

	Opd	Dtype	Shape	Layout
源操作数	a	INT8/INT16/INT32/FP16 /BF16/FP32/FP8	(n, c, h, w) 或 (n/1, c, h/1, w/1)	任意排列/ 常数
目的操作数	out	同 a	(n, c, h, w)	任意排列

3.6.16 浮点融合计算指令

```
rvt_fmuladd out, a, b, c  
rvt_fsqradd out, a, b  
rvt_fsqrsub out, a, b
```

3.6.16.1 rvt_fmuladd

```
rvt_fmuladd out, a, b, c
```

浮点乘加指令。指令将 Tensor A 和 B 相乘并与 Tensor C 相加，并将结果写入到 Tensor Out 中去。并且支持：

- 支持保存结果时是否饱和，由 saturate 控制；

指令的计算逻辑如下所示：

```
out = a * b + c;
```

指令各操作数的约束如下所示：

	Opd	Dtype	Shape	Layout
源操作数	a	BF16/FP16/FP32	(n, c, h, w)	HW 对齐排列/常数
	b	同 a	(1, c, 1, 1)	紧密排列/常数
	c	同 a	(1, c, 1, 1)	紧密排列/常数
目的操作数	out	同 a	(n, c, h, w)	HW 对齐排列

3.6.16.2 rvt_fsqradd

```
rvt_fsqradd out, a, b
```

浮点加的平方指令。指令将 Tensor A 和 B 相加，并将结果做平方计算。并且支持：

- 支持保存结果时是否饱和，由 saturate 控制；

指令的计算逻辑如下所示：

```
out = (a + b)**2;
```

指令各操作数的约束如下所示：

	Opd	Dtype	Shape	Layout
源操作数	a	BF16/FP16/FP32	(n, c, h, w)	HW 对齐排列/常数
	b	同 a	(1, c, 1, 1)	紧密排列/常数
目的操作数	out	同 a	(n, c, h, w)	HW 对齐排列

3.6.16.3 rvt_fsqrsub

```
rvt_fsqrsub out, a, b
```

浮点差的平方指令。指令将 **Tensor A** 和 **B** 相减，并将结果做平方计算。并且支持：

- 支持保存结果时是否饱和，由 `saturate` 控制；

指令的计算逻辑如下所示：

```
out = (a - b)**2;
```

指令各操作数的约束如下所示：

	Opd	Dtype	Shape	Layout
源操作数	a	BF16/FP16/FP32	(n, c, h, w)	HW 对齐排列/常数
	b	同 a	(1, c, 1, 1)	紧密排列/常数
目的操作数	out	同 a	(n, c, h, w)	HW 对齐排列

3.6.17 整数量化与反量化指令

对 Tensor A 和 B 进行逐运算比较，根据比较结果选择 Tensor C 或 0 的值写入到 Tensor Out 中去。指令支持 INT8/INT16/INT32 等精度。具体指令如下：

```
rvt_dq0 out,a,scale
rvt_rq0 out,a,scale
rvt_dq1 out,a,scale
rvt_rq1 out,a,scale
```

3.6.17.1 rvt_dq0

```
rvt_dq0 out,a,scale
```

反量化指令。指令使用 Tensor Scale 对 Tensor A 做缩放，并将结果转换为浮点数。该指令还支持：

- 支持调整整型转浮点的 Rounding 模式，由 CSR.Ctrl.round_mode 控制；
- 支持对浮点结果做饱和；

指令的计算逻辑如下所示：

```
_scale, _offset = scale
ix = x - _offset;
fx = IntToFloat(ix, FP32, CSR.Ctrl.round_mode);
out = fx * scale;
```

指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout	Note
源操作数	x	S4/U4/S8/U8/S16/U16	(n, ic, ih, iw)	HW 对齐排列	输入 Tensor
	scale	INT32	(1, oc, 1, 2)	紧密排列或常数	反量化参数 Tensor (0,c,0,0[15:0]) 为 offset(S16); (0,c,0,1) 为 scale(FP32); 当为常数时： CR.val[31:0]为 scale(FP32); CR.val[47:16]为 offset(S16);
目的操作数	out	BF16/FP32	(n, oc, oh, ow)	HW 对齐排列	输出 Tensor

3.6.17.2 rvt_rq0

```
rvt_rq0 out,a,scale
```

量化指令。指令使用 Tensor Scale 对 Tensor A 做缩放，并将结果转为低精度的整型。该指令还支持：

- 支持整型饱和；

- 支持调整浮点转为整型的 Rounding 模式，由 CSR.Ctrl.round_mode 控制；
- 指令的计算逻辑如下所示：

```

_scale, _offset = scale
fx = IntToFloat(x, FP32, RM_HALF_AWAY_FROM_ZERO);
fy = scale * x + offset;
iy = FloatToInt(fy, out.teew, out.subtype,
                CSR.ctrl.round_mode);

```

指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout	Note
源操作数	x	S8/U8/S16/U16/S32/U32	(n, ic, ih, iw)	HW 对齐排列	输入 Tenor
	scale	U32	(1, oc, 1, 2)	紧密排列	反量化参数 Tensor (0,c,0,0) 为 scale(FP32); (0,c,0,1) 为 offset(FP32); 当为常数时： CR.val[31:0]为 offset; CR.val[55:32]为(scale >> 8);
目的操作数	out	BF16/FP32	(n, oc, oh, ow)	HW 对齐排列	输出 Tensor

3.6.17.3 rvt_dq1

```

rvt_dq1 out,a,scale

```

反量化指令。指令使用 Tensor Scale 对 Tensor A 做缩放，并将结果转为高精度的整型。且指令还支持：

- 支持右移的 Rounding 模式，由 CSR.Ctrl.round_mode 控制；
- 支持整型饱和；

指令的计算逻辑如下所示：

```

multiplier, shift, offset = scale;
iy = (x - offset) * multiplier;
iy = arith_shift_round(iy, shift, CSR.ctrl.round_mode);
out = satu(iy, out.teew, out.subtype);

```

指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout	Note
源操作数	x	S4/U4/S8/U8/S16/U16/	(n, ic, ih, iw)	HW 对齐排列	输入 Tenor

	scale	U32	(1, oc, 1, 2)	紧密排列 或常数	量化参数 Tensor (1,c,1,0[31:0]) 为 multiplier(S32); (1,c,1,1[7:0]) 为 shift(S8); (1,c,1,1[31:16]) 为 offset(S16); 当为常数时: CR.val[31:0]为 multiplier(S32); CR.val[39:32]为 shift(S8); CR.val[55:40]为 offset(S16);
目的 操作数	out	S8/U8/S16/U16/ S32/U32	(n, oc, oh, ow)	HW 对齐排列	输出 Tensor

3.6.17.4 rvt_rq1

```
rvt_rq1 out, a, scale
```

量化指令。指令使用 Tensor Scale 对 Tensor A 做缩放，并将结果转为低精度的整型。且指令还支持：

- 支持右移的 Rounding 模式，由 CSR.Ctrl.round_mode 控制；
- 支持整型饱和；

指令的计算逻辑如下所示：

```
multiplier, shift, offset = scale;
iy = x * multiplier;
iy = arith_shift_round(iy, shift, CSR.ctrl.round_mode);
iy += offset;
out = satu(iy, out.teew, out.subtype);
```

指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout	Note
源 操作数	x	S8/U8/S16/U16/ S32/U32	(n, c, h, w)	HW 对齐排列	输入 Tensor
	scale	U32	(1, oc, 1, 2)	紧密排列 或常数	量化参数 Tensor (1,c,1,0[31:0]) 为 multiplier(S32); (1,c,1,1[7:0]) 为 shift(S8); (1,c,1,1[31:16]) 为 offset(S16); 当为常数时: CR.val[31:0]为 multiplier(S32); CR.val[39:32]为 shift(S8); CR.val[55:40]为 offset(S16);
目的 操作数	out	S4/U4/S8/U8/ S16/U16/	(n, c, h, w)	HW 对齐排列	输出 Tensor

3.6.18 SFU 指令

```
rvt_sfu_norm out, a
rvt_sfu_taylor out, a, coeff
rvt_sfu_rsqrtr out, a
```

3.6.18.1 rvt_sfu_norm

```
rvt_sfu_norm out, a
```

提取浮点数指数部分指令。指令对输入浮点 Tensor A 提取指数部分并转化为整型输出到 Tensor out 中。指令的计算逻辑如下所示：

```
if (a.teew == e32)
    exp_inc = a.frac>0x2aaaab && a>0;
else if (a.teew == e16)
    exp_inc = a.frac>0x155 && a>0;
else
    exp_inc = a.frac>0x2b && a>0;
out = a.exp + exp_inc;
```

指令各操作数的约束如下所示：

	Opd	Dtype		shape	layout
源操作数	a	FP16/BF16	FP32	(n, c, h, w)	HW 对齐排列
目的操作数	out	INT16	INT32	(n, c, h, w)	HW 对齐排列

3.6.18.2 rvt_sfu_taylor

```
rvt_sfu_taylor out, a, coeff
```

多项式计算指令。指令对输入浮点 Tensor A 和 Coeff 做多项式计算，并输出结果到 Tensor out 中。指令的计算逻辑如下所示：

```
out = coeff[coeff.w-1];
for (i=coeff.w-1; i>0; i--)
    out = out*a + coeff[i-1];
```

指令各操作数的约束如下所示：

	Opd	Dtype		shape	layout
源操作数	a	FP16/BF16/FP32		(n, c, h, w)	HW 对齐排列
	coeff	同 a		(1, lane_num, 1, wb)	紧密排列
目的操作数	out	同 a		(n, c, h, w)	HW 对齐排列

3.6.18.3 rvt_sfu_rsqr

```
rvt_sfu_rsqr out, a
```

浮点倒数平方根指令。指令对输入浮点 Tensor A 求倒数平方根计算，并输出结果到 Tensor out 中。并且指令支持：

- 支持做牛顿迭代时的迭代次数，由 CSR.Ctrl.rsqr_iter 控制。

指令的计算逻辑如下所示：

```
out = coeff[coeff.w-1];
for (i=coeff.w-1;i>0;i--)
    out = out*+ ceoff[i-1];
x = a;
xhalf = 0.5*x;
ix = *((int*)&x);
i = CONST_MAGIC_NUM - (i >> 1);
x = *(float *)&i;
while(CSR.ctrl.rsqrt_iter--) {
    x = x*(1.5-xhalf*x*x);
}
out = x;
```

指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout
源操作数	a	FP16/BF16/FP32	(n, c, h, w)	HW 对齐排列
目的操作数	out	同 a	(n, c, h, w)	HW 对齐排列

3.6.19 Gather&Scatter 指令

rvt_gather_pc	out, a, idx, cs
rvt_scatter_pc	out, a, idx
rvt_gather	out, a, idx, cs, _hzd
rvt_scatter	out, a, idx, _hzd
rvt_hgather	out, a, idx
rvt_hscatter	out, a, idx
rvt_masksel	out_cnt, out, a, mask, _hzd
rvt_nzidx	out_cnt, out_idx, a, _hzd

3.6.19.1 rvt_gather_pc

rvt_gather_pc	out, a, idx, cs, _bdlimit
---------------	---------------------------

Gather 指令。指令按照所有 Channel 共享 Index 的方式，将 Tensor A 的 W 维的数据 gather 到 Tensor Out 中。

指令的计算逻辑如下所示：

```
for(n,c,w in out.shape) {  
    w_idx = idx(0,w,0,0);  
    if (w_idx < a.W)  
        out(n,c,0,w) = a(n,c,0,w_idx);  
    else  
        out(n,c,0,w) = cs.val;  
}
```

指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout
源操作数	a	e8/e16/e32	(n, c, 1, wa)	HW 对齐排列
	idx	U8/U16	(1, w, 1, 1)	紧密排列
	cs	同 a	--	常数
目的操作数	out	同 a	(n, c, 1, w)	HW 对齐排列

3.6.19.2 rvt_scatter_pc

rvt_scatter_pc	out, a, idx, _bdlimit
----------------	-----------------------

Scatter 指令。指令按照所有 Channel 共享 Index 的方式，将 Tensor A 的 W 维的数据 scatter 到 Tensor Out 中。

指令的计算逻辑如下所示：

```
for(n,c,w in a.shape) {  
    w_idx = idx(0,w,0,0);  
    if (w_idx < out.w)  
        out(n,c,0,w_idx) = a(n,c,0,w);  
}
```

指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout
源操作数	a	e8/e16/e32	(n, c, 1, w)	HW 对齐排列

	idx	U8/U16	(1, w, 1, 1)	紧密排列
目的操作数	out	同 a	(n, c, 1, wo)	HW 对齐排列

3.6.19.3 rvt_gather

```
rvt_gather out, a, idx, cs, _bdlimit, _hzd
```

Gather 指令。指令按照 Index 将 Tensor A 的 W 维的数据 gather 到 Tensor Out 中。

此指令还支持：

- 当 out, a, idx 存在 bank 冲突时，需将_hzd 设置为 1。

指令的计算逻辑如下所示：

```
for(n, c, w in idx.shape) {
    w_idx = idx(n, c, 0, w);
    if(w_idx < a.w)
        out(n, c, 0, w) = a(n, c, 0, w_idx);
    else
        out(n, c, 0, w) = cs.val;
}
```

指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout
源操作数	x	e8/e16/e32	(1, c, 1, wa)	HW 对齐排列
			(1, 1, 1, wa)	紧密排列
	idx	U8/U16	(n, c, 1, w)	紧密排列
	cs	同 x	--	常数
目的操作数	out	同 x	(n, c, 1, w)	HW 对齐排列

3.6.19.4 rvt_scatter

```
rvt_scatter out, a, idx, _bdlimit, _hzd
```

Scatter 指令。指令按照 Index 将 Tensor A 的 W 维的数据 scatter 到 Tensor Out 中。此指令还支持：

- 当 out, a, idx 存在 bank 冲突时，需将_hzd 设置为 1。

指令的计算逻辑如下所示：

```
for(n, c, w in idx.shape) {
    w_idx = idx(n, c, 0, w);
    if (w_idx < out.w)
        out(n, c, 0, w_idx) = a(n, c, 0, w);
}
```

指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout
源操作数	x	e8/e16/e32	(1, c, 1, w)	HW 对齐排列
			(1, 1, 1, w)	紧密排列
	idx	U8/U16	(n, c, 1, w)	紧密排列
目的操作数	out	同 x	(n, c, 1, wo)	HW 对齐排列

3.6.19.5 rvt_hgather

`rvt_hgather` `out,a,idx,cs,_bdlimit`

➤ 行 Gather 指令。指令按照 Idx 将 Tensor A 的 H 维的数据 gather 到 Tensor Out 中。

指令的计算逻辑如下所示：

```
for(n,c in idx.shape) {
    h_idx = idx(n,c,h,0);
    if (h_idx < a.h)
        out(n,c,h,:) = a(n,c,h_idx,:);
    else
        out(n,c,h,:) = cs.val
}
```

指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout
源操作数	x	e8/e16/e32	(1, c, ha, w)	行对齐排列
	idx	U8/U16	(n, c, h, 1)	HW 对齐排列
	cs	同 x	--	常数
目的操作数	out	同 x	(n, c, h, w)	行对齐排列

3.6.19.6 rvt_hscatter

`rvt_hscatter` `out,a,idx,_bdlimit`

行 Scatter 指令。指令按照 Idx 将 Tensor A 的 H 维的数据 scatter 到 Tensor Out。

指令的计算逻辑如下所示：

```
for(n,c in idx.shape) {
    h_idx = idx(n,c,h,0);
    if (h_idx < out.h)
        out(n,c,h_idx,:) = a(n,c,h,:);
}
```

指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout
源操作数	x	e8/e16/e32	(1, c, h, w)	行对齐排列
	idx	U8/U16	(n, c, h, 1)	HW 对齐排列
目的操作数	out	同 x	(n, c, ho, w)	行对齐排列

3.6.19.7 rvt_masksel

`rvt_masksel` `out_cnt,out,a,mask,_hzd`

Mask 选择指令。指令将 Mask 为 1 相同位置上的 Tensor A 的 W 维元素写到 Tensor Out 中。此指令还支持：

- 返回每个 channel 的元素的个数并写到 Tensor Out_cnt 中；
- 当 out, a, mask 存在 bank 冲突时，需将_hzd 设置为 1。

指令的计算逻辑如下所示：

```
for(n,c in out.shape.n,out.shape.c) {
    out_cnt(n,c) = 0;
    for(wa in a.shape.w) {
        msk = mask(n,c,0,wa);
        if(msk==1){
            out(n,c,0,out_cnt(n,c))=(0,c,0,wa);
            out_cnt(n,c) += 1;
        }
    }
}
```

指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout
源操作数	a	e8/e16/e32	(1, c, 1, w)	HW 对齐排列
			(1, 1, 1, w)	紧密排列
	mask	U8/U16/U32	(n, c, 1, w)	HW 对齐排列
目的操作数	out	同 a	(n, c, 1, w)	HW 对齐排列
	out_cnt	U16	(n, c, 1, 1)	紧密排列

3.6.19.8 rvt_nzidx

```
rvt_nzidx          out_cnt,out_idx,a,_hzd
```

生成非 0 元素索引的指令。指令 Tensor A 的 W 维非 0 元素的索引写到 Tensor Out 中。此指令还支持：

- 返回每个 channel 的非 0 元素的个数并写到 Tensor Out_cnt 中；
- 当 out, a 存在 bank 冲突时，需将_hzd 设置为 1。

指令的计算逻辑如下所示：

```
for(n,c in out.shape.n,out.shape.c) {
    out_cnt(n,c) = 0;
    for(wa in a.shape.w) {
        data = a(n,c,0,wa);
        if(data!=0){
            out(n,c,0,out_cnt(n,c))= wa;
            out_cnt(n,c) += 1;
        }
    }
}
```

指令各操作数的约束如下所示：

	Opd	Dtype	Shape	Layout
源操作数	a	e8/e16/e32	(n, c, 1, w)	HW 对齐排列
目的操作数	out	同 a	(n, c, 1, w)	HW 对齐排列
	out_cnt	U16	(n, c, 1, 1)	紧密排列

3.6.20 CW 转置指令

```
rvt_trans_cw out, a  
rvt_trans_wc out, a
```

3.6.20.1 rvt_trans_cw

CW 维转置指令。指令对输入浮点 Tensor A 的 C 和 W 维做转置，并输出结果到 Tensor out 中。指令的计算逻辑如下所示：

```
out(n, w, h, c) = a(n, c, h, w);
```

指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout
源操作数	a	S8/U8/S16/U16/S32/U32/ FP16/BF16/FP32	(n, c, h, w)	HW 对齐排列
目的操作数	out	同 a	(n, c, h, w)	HW 行对齐排列

3.6.20.2 rvt_trans_wc

WC 维转置指令。指令对输入浮点 Tensor A 的 W 和 C 维做转置，并输出结果到 Tensor out 中。指令的计算逻辑如下所示：

```
out(n, w, h, c) = a(n, c, h, w);
```

指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout
源操作数	a	S8/U8/S16/U16/S32/U32/ FP16/BF16/FP32	(n, c, h, w)	HW 对齐排列
目的操作数	out	同 a	(n, c, h, w)	HW 对齐排列

3.6.21 SMEM 数据加载指令

将 SMEM 的数据广播或按 lane 分拆到 LMEM 中。指令支持 8/16/32 bit 等精度。

具体指令如下：

```
rvt_smem_bc    out, (rs)
rvt_smem_dist  out, (rs)
```

3.6.21.1 rvt_smem_bc

```
rvt_smem_bc    out, (rs)
```

SMEM 数据广播指令。指令将 rs 地址指向的 SMEM 的数据复制到每个 LANE 上。指令的计算逻辑如下所示：

```
ptr = (rs);
for(c,w in out.shape) {
    out(0,c,0,w) = ptr[w];
}
```

指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout	Note
源操作数	rs	地址(相对地址)	--	--	
目的操作数	out	e8/ e16/e32	(1, c, 1, w)	HW 对齐排列	c 不大于 LANE_NUM

3.6.21.2 rvt_smem_dist

```
rvt_smem_dist  out, (rs)
```

SMEM 数据分散指令。指令将 rs 地址指向的 SMEM 的数据分散到每个 LANE 上。指令的计算逻辑如下所示：

```
ptr = (rs);
for(c,w in td.shape) {
    out(0,c,0,0) = ptr[c];
}
```

指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout
源操作数	rs	地址(相对地址)	--	--
目的操作数	out	e8/ e16/e32	(1, c, 1, 1)	紧密排列

3.6.22 LANE 复制指令

```
rvt_lane_cp out,a
rvt_lane_bc out,a
```

3.6.22.1 rvt_lane_cp

```
rvt_lane_cp out,a
```

LANE 复制指令。指令将 Tensor A 跨 LANE 复制到 Tensor out 中。指令的计算逻辑如下所示：

```
out(n,c+lane_offset,h,w) = a(n,c,h,w);
```

注意：

➤ Tensor Out 的起始 LANE ，体现在其在 LMEM 的起始地址 offset 上。

指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout
源操作数	a	e8/e16/e32	(n, c, h, w)	HW 对齐排列
目的操作数	out	同 a	(n, c, h, w)	HW 对齐排列

3.6.22.2 rvt_lane_bc

```
rvt_lane_bc out,a
```

LANE 广播指令。指令将 Tensor A 广播到其他 LANE 上。指令的计算逻辑如下所示：

```
out(n,c,h,w) = a(n,0,h,w);
```

指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout
源操作数	a	e8/e16/e32	(n, 1, h, w)	HW 对齐排列
目的操作数	out	同 a	(n, c, h, w)	HW 对齐排列

3.6.23 消息同步指令

通过消息同步机制进行多核间的同步操作。具体指令如下：

```
rvt_msgsend    rs  
rvt_msgwait    rs
```

3.6.23.1 rvt_msgsend

```
rvt_msgsend    rs
```

消息发送指令。rs 包含 message ID 以及等待该消息者的个数。指令的计算逻辑如下所示：

```
msg_id = rs[15:0];  
wait_cnt = rs[31:16];  
msg_que[msg_id].remain_wait_cnt = wait_cnt;  
msg_que[msg_id].sent_cnt++;
```

3.6.23.2 rvt_msgwait

```
rvt_msgwait    rs
```

消息发送指令。rs 包含 message ID 以及等待该消息者的个数。指令的计算逻辑如下所示：

```
msg_id = rs[15:0];  
send_cnt = rs[31:16];  
while(msg_que[msg_id].sent_cnt != send_cnt)  
    continue;  
if(--msg_que[msg_id].remain_wait_cnt == 0)  
    msg_que[msg_id].send_cnt = 0;
```

3.7 DMA 指令

3.7.1 Tensor 数据加载指令

从 GMEM 加载 Tensor 到 LMEM 中，支持 8/16/32 bit 等数据位宽。具体指令如下：

```
rvt_dma_ld    td,gs
rvt_dma_ldbc  td,gs
rvt_dma_ldt   td,gs
```

3.7.1.1 rvt_dma_ld

```
rvt_dma_ld    td,gs
```

DMA 张量加载指令。指令将源张量从 GMEM 加载到 LMEM 中。指令的计算逻辑如下所示：

```
td(on,oc,oh,ow) = gs(in,ic,ih,iw);
```

注意：

➤ 源 Tensor 和目的 Tensor 的 Shape 可以不一样，但元素总量须保持一致；

指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout	Opd Type
源操作数	gs	e8/e16/e32	(in, ic, ih, iw)	任意排列	GR
目的操作数	td	同 a	(on, oc, oh, ow)	任意排列	TR

3.7.1.2 rvt_dma_ldbc

```
rvt_dma_ldbc  td,gs
```

DMA 张量加载广播指令。指令将源张量从 GMEM 加载并广播到 LMEM 中。指令的计算逻辑如下所示：

```
td(n,c,h,w) = gs(n,1,h,w);
```

注意：

➤ 源 Tensor 和目的 Tensor 的 Shape 除 C 维外其他维度大小须保持一致；

➤ 源 Tensor 的 C 维须为 1，且广播时不支持 LANE 绕回，也不支持 LaneMask；

指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout	Opd Type
源操作数	gs	e8/e16/e32	(n, 1, h, w)	任意排列	GR
目的操作数	td	同 a	(n, c, h, w)	任意排列	TR

3.7.1.3 rvt_dma_ldt

```
rvt_dma_ldt   td,gs
```

DMA 张量转置加载指令。指令将源张量从 GMEM 做 NC 转置后加载到 LMEM 中。指令的计算逻辑如下所示：

```
td(c,n,h,w) = gs(n,c,h,w);
```

注意：

➤ 源 Tensor 和目的 Tensor 的 H 和 W 大小须保持一致；

指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout	Opd Type
源操作数	gs	e8/e16/e32	(in, ic, ih, iw)	任意排列	GR
目的操作数	td	同 a	(on, oc, oh, ow)	任意排列	TR

3.7.2 Tensor 数据存储指令

将在 LMEM 中的 Tensor 存储到 SMEM 中，支持 8/16/32 bit 等数据位宽。具体指令如下：

```
rvt_dma_st      gd, ts
rvt_dma_stt     gd, ts
rvt_dma_stred   gd, ts
```

3.7.2.1 rvt_dma_st

```
rvt_dma_st      gd, ts
```

DMA 张量存储指令。指令将源张量从 LMEM 存储到 GMEM 中。指令的计算逻辑如下所示：

```
if(is_TR(ts))
    gd(on, oc, oh, ow) = ts(in, ic, ih, iw);
else
    gd(on, oc, oh, ow) = ts.val;
```

注意：

- 源 Tensor 和目的 Tensor 的 Shape 可以不一样，但元素总量须保持一致；
- 源操作数可以为 CR，表示填充常数到目的 Tensor 中。

指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout	Opd Type
源操作数	ts	e8/e16/e32	(in, ic, ih, iw)	任意排列 或常数	TR/CR
目的操作数	gd	同 a	(on, oc, oh, ow)	任意排列	GR

3.7.2.2 rvt_dma_stt

```
rvt_dma_stt     gd, ts
```

DMA 张量转置存储指令。指令将源张量从 LMEM 做 NC 转置后存储到 GMEM 中。指令的计算逻辑如下所示：

```
gd(c, n, h, w) = ts(n, c, h, w);
```

注意：

- 源 Tensor 和目的 Tensor 的 H 和 W 大小须保持一致；

指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout	Opd Type
源操作数	ts	e8/e16/e32	(n, c, h, w)	任意排列	TR
目的操作数	gd	同 a	(c, n, h, w)	任意排列	GR

3.7.3 数据复制指令

将 Tensor 在各个 memory 间进行复制，支持常数填充、线性复制、广播等操作，支持 8/16/32 bit 等数据位宽。具体指令如下：

```
rvt_dma_cp    dst,src  
rvt_dma_sbc   td,src
```

3.7.3.1 rvt_dma_cp

```
rvt_dma_cp    dst,src
```

DMA 张量复制指令。指令将张量在 LMEM 或 GMEM 内部做搬运。指令的计算逻辑如下所示：

```
dst(on,oc,oh,ow) = src(in,ic,ih,iw);
```

注意：

- 源 Tensor 和目的 Tensor 的 Shape 可以不一样，但元素总量须保持一致；
- 指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout	Opd Type
源操作数	src	e8/e16/e32	(in, ic, ih, iw)	任意排列	TR/GR
目的操作数	dst	同 src	(on, oc, oh, ow)	任意排列	TR/GR

3.7.3.2 rvt_dma_sbc

```
rvt_dma_sbc   td,src
```

DMA 张量广播指令。指令将张量作为 1 维数据在 GMEM/SMEM 和 LMEM 之间或 LMEM 内部做广播搬运。指令的计算逻辑如下所示：

```
dst(0,c,0,n*h*w) = src(0,0,0,n*h*w);
```

指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout	Opd Type
源操作数	src	e8/e16/e32	(n, 1, h, w)	紧密排列	TR/GR
目的操作数	dst	同 src	(n, c, h, w)	紧密排列	TR

3.7.3.3 rvt_dma_red

```
rvt_dma_red    gd, ts, _rw, _op
```

DMA 张量存储并做 reduce 的指令。指令将源张量从 LMEM 存储到 L2M 后，再与原有的数据做 reduce 计算操作。指令的计算逻辑如下所示：

```
gd(n,c,h,w) = _red(gd(n,c,h,w), ts(n,c,h,w),_rw,_op);
```

注意：

- 源 Tensor 和目的 Tensor 的 Shape 可以不一样，但元素总量须保持一致；

➤ 仅支持目的地址为 L2M

指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout	Opd Type
源操作数	ts	e8/e16/e32	(in, ic, ih, iw)	任意排列	TR
目的操作数	gd	同 a	(on, oc, oh, ow)	任意排列	GR

3.7.4 转置指令

将 Tensor 数据在 GMEM 或 LMEM 中进行 NC 维度或 CW 维度的转置，支持 8/16/32 bit 等数据位宽。具体指令如下：

```
rvt_dma_nctrans    dst,src
```

3.7.4.1 rvt_dma_nctrans

```
rvt_dma_nctrans    dst,src
```

DMA NC 转置指令。指令将张量在 GMEM 或 LMEM 内部做 NC 转置搬运。指令的计算逻辑如下所示：

```
dst(c,n,h,w) = src(n,c,h,w);
```

指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout	Opd Type
源操作数	src	e8/e16/e32	(n, c, h, w)	任意排列	TR/GR
目的操作数	dst	同 src	(c, n, h, w)	任意排列	同 src

3.7.5 倒置指令

```
rvt_dma_reverse dst,src,_dim
```

3.7.5.1 rvt_dma_reverse

```
rvt_dma_reverse dst,src,_dim
```

将 Tensor 数据沿着_dim 维度做倒置，支持 8/16/32 bit 等数据位宽。指令的计算逻辑如下所示：

```
size = src.shape[dim];
for(i in size) {
    if(dim==3):
        dst(i,c,h,w) = src(size-1-i,c,h,w);
    else if(dim == 2):
        dst(n,i,h,w) = src(n,size-1-i,h,w);
    else if(dim == 1):
        dst(n,c,i,w) = src(n,c,size-1-i,w);
    else if(dim == 0):
        dst(n,c,h,i) = src(n,c,h,size-1-i);
}
```

注意：

- 当在 src mem 为 LMEM 时只支持 c 维的反转;

指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout	Opd Type
源操作数	src	e8/e16/e32	(n, c, h, w)	任意排列	TR/GR
目的操作数	dst	同 src	(c, n, h, w)	任意排列	TR/GR

3.7.6 Mask&Index 指令

根据 mask 从 src 中筛选数据并写到 dst 中，支持 8/16/32 bit 等数据位宽。具体指令如下：

```
rvt_dma_masksel    dst,src,mask
rvt_dma_fmasksel   dst,src,mask
rvt_dma_nzidx       dst,src
rvt_dma_fnzidx      dst,src
```

3.7.6.1 rvt_dma_masksel

```
rvt_dma_masksel    dst,src,mask
```

整型 Mask 选择指令。指令将 Tensor src 中与 Mask 中相同位置上值为 1 的元素挑选出来，并输出到 Tensor Dst 中。此指令还支持：

- 输出收集到的 Dst 的元素个数到 filter_num 寄存器；

指令的计算逻辑如下所示：

```
cnt = 0;
for(n,c,h,w in src.shape) {
    if(0 == mask(n,c,h,w)
        continue;
    dst[cnt++] = src(n,c,h,w);
}

filter_num = cnt;
```

	Opd	Dtype	shape	layout	Opd Type
源操作数	src	S8/U8/S16/ U16/S32/U32	(n, c, h, w)	在 LMEM 时 HW 对齐 排列否则紧密排列	TR/GR
	mask	U32	(n, c, h, w)	在 LMEM 时 HW 对齐 排列否则紧密排列	TR/GR
目的操作数	dst	同 src	(max_filtered_size,)	紧密排列	GR

3.7.6.2 rvt_dma_fmasksel

```
rvt_dma_fmasksel   dst,src,mask
```

浮点 Mask 选择指令。指令将 Tensor src 中与 Mask 中相同位置上值为 1 的元素挑选出来，并输出到 Tensor Dst 中。此指令还支持：

- 输出收集到的 Dst 的元素个数到 filter_num 寄存器；

指令的计算逻辑如下所示：

```

cnt = 0;
for(n,c,h,w in src.shape) {
    if(0 == mask(n,c,h,w)
        continue;
    dst[cnt++] = src(n,c,h,w);
}

filter_num = cnt;

```

	Opd	Dtype	shape	layout	Opd Type
源操作数	src	BF16/FP16/ FP32	(n, c, h, w)	在 LMEM 时 HW 对齐 排列否则紧密排列	TR/GR
	mask	U32	(n, c, h, w)	在 LMEM 时 HW 对齐 排列否则紧密排列	TR/GR
目的操作数	dst	同 src	(max_filtered_size,)	紧密排列	GR

3.7.6.3 rvt_dma_nzidx

```
rvt_dma_nzidx dst,src
```

整型非 0 数索引生成指令。指令生成 Tensor src 中非 0 元素的索引，并输出到 Tensor Dst 中。此指令还支持：

- 支持生成索引的起始值，由 CSR.Dma_Idx.start_pos 控制；
- 输出生成的索引个数到 filter_num 寄存器；

指令的计算逻辑如下所示：

```

cnt = 0
idx = 0;
for(n,c,h,w in src.shape) {
    if(0 == mask(n,c,h,w)) {
        idx++;
        continue;
    }
    dst[cnt++] = idx++;
}
filter_num = cnt;

```

指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout	Opd Type
源操作数	src	S8/U8/S16/U16/ S32/U32	(n, c, h, w)	在 LMEM 时 HW 对齐排列否 则紧密排列	TR/GR
目的操作数	dst	U8/U16/U32	(n, c, h, w)	紧密排列	GR

3.7.6.4 rvt_dma_fnzidx

```
rvt_dma_fnzidx dst,src
```

浮点非 0 数索引生成指令。指令生成 Tensor src 中非 0 元素的索引，并输出到

Tensor Dst 中。此指令还支持：

- 支持生成索引的起始值，由 CSR.Dma_Idx.start_pos 控制；
- 输出生成的索引个数到 filter_num 寄存器；

指令的计算逻辑如下所示：

```
cnt = 0
idx = 0;
for(n,c,h,w in src.shape) {
    if(0 == mask(n,c,h,w)) {
        idx++;
        continue;
    }
    dst[cnt++] = idx++;
}
filter_num = cnt;
```

指令各操作数的约束如下所示：

	Opd	Dtype	shape	layout	Opd Type
源操作数	src	FP16/BF16/FP32	(n, c, h, w)	在 LMEM 时 HW 对齐排列否则紧密排列	TR/GR
目的操作数	dst	U8/U16/U32	(n, c, h, w)	紧密排列	GR

3.7.7 Gather&Scatter 指令

根据 Index 从 src 中收集或分散数据到 dst 中，支持 8/16/32 bit 等数据位宽。具体指令如下：

```
rvt_dma_hgather    dst,src,idx
rvt_dma_hscatter    dst,src,idx
rvt_dma_fhscatter    dst,src,idx
```

3.7.7.1 rvt_dma_hgather

```
rvt_dma_hgather    dst,src,idx
```

行 Gather 指令。指令按照 Idx 将 Tensor A 的 H 维的数据 gather 到 Tensor Out 中。此指令还支持：

- 支持设置 Idx 的 start position，由 CSR.Dma_Idx.start_pos 控制；
- 支持当 Idx 的值小于 start_pos 时，输出默认的常数值，由 CSR.Dma_Idx.default_const_val 控制。

指令的计算逻辑如下所示：

```
for(c, h in dst.shape) {
    h_idx = idx(0, (idx.c==1)?0:c,h,0);
    if (h_idx < CSR.dma_idx.start_pos) {
        dst(0,c,h,:) = 0;
        continue;
    }
    h_idx -= start_pos;
    dst(0,c,h,:) = src(0, (src.c==1)?0:c,h_idx,:);
}
```

指令各操作数的约束如下所示，注意：src 和 idx 的 c 不能同时为 1：

	Opd	Dtype	shape	layout	Opd Type
源操作数	src	e8/e16/e32	(1, c, ih, w) 或 (1, 1, ih, w)	任意排列但 stride_w 须为 1	TR/GR
	idx	U32	(1, c, h, 1) 或 (1, 1, h, 1)	任意排列但 stride_h 和 stride_w 须为 1	TR/GR
目的操作数	dst	同 src	(1, c, h, w)	任意排列但 stride_w 须为 1	TR/GR

3.7.7.2 rvt_dma_hscatter

```
rvt_dma_hscatter    dst,src,idx,_accu
```

整型行 Scatter 指令。指令按照 Idx 将 Tensor A 的 H 维的数据 scatter 到 Tensor Out 中。此指令还支持：

- 支持设置 Idx 的 start position，由 CSR.Dma_Idx.start_pos 控制，当 Idx 小于 start_pos 时跳过当次 scatter 操作；

- 支持将 scatter 值和 Tensor Out 原有的值做累加，由 _accu 控制，仅支持数据类型为 U32 时；

指令的计算逻辑如下所示：

```
for(c in out.shape.c)
  for (ha in src.shape.h) {
    h_idx = idx(0,idx.c==1?0:c,ha,0);
    if (h_idx < CSR.dma_idx.start_pos) {
      continue;
    }
    h_idx -= start_pos;
    if(_accu)
      dst(0,c,h_idx,:) += src(0,src.c==1?0:c,ha,:);
    else
      dst(0,c,h_idx,:) = src(0,src.c==1?0:c,ha,:);
  }
}
```

指令各操作数的约束如下所示，注意 src 和 idx 的 c 不能同时为 1：

	Opd	Dtype	shape	layout	Opd Type
源操作数	src	U8/U16/ U32	(1, c, ih, w) 或 (1, 1, ih, w)	任意排列但 stride_w 须为 1	TR/GR
	idx	U32	(1, c, ih, 1) 或 (1, 1, ih, 1)	任意排列但 stride_h 和 stride_w 须 为 1	TR/GR
目的操作数	dst	同 src	(1, c, oh, w)	任意排列但 stride_w 须为 1	TR
			(c, oh_1, oh_0, w)	Free Mode 但 stride_w 须为 1 且 stride_n = (ih_1*ih_0)*stride_h	GR

3.7.7.3 rvt_dma_fhscatter

```
rvt_dma_fhscatter dst,src,idx,_high,_accu
```

浮点行 Scatter 指令。指令按照 Idx 将 Tensor A 的 H 维的数据 scatter 到 Tensor Out 中。此指令还支持：

- 支持设置 Idx 的 start position，由 CSR.Dma_Idx.start_pos 控制，当 Idx 小于 start_pos 时跳过当次 scatter 操作；
- 支持将 scatter 值和 Tensor Out 原有的值做累加，由 _accu 控制；

指令的计算逻辑如下所示：

```
for(c in out.shape.c)
  for (ha in src.shape.h) {
    h_idx = idx(0,idx.c==1?0:c,ha,0);
    if (h_idx < CSR.dma_idx.start_pos) {
      continue;
    }
    h_idx -= start_pos;
    if(_accu)
      dst(0,c,h_idx,:) += src(0,src.c==1?0:c,ha,:);
    else
```



```
        dst(0,c,h_idx,:) = src(0,src.c==1?0:c,ha,:);
    }
}
```

指令各操作数的约束如下所示，注意：src 和 idx 的 c 不能同时为 1：

	Opd	Dtype	shape	layout	Opd Type
源操作数	src	FP16/BF16/ FP32	(1, c, ih, w) 或 (1, 1, ih, w)	任意排列但 stride_w 须为 1	TR/GR
	idx	U32	(1, c, ih, 1) 或 (1, 1, ih, 1)	任 意 排 列 但 stride_h 和 stride_w 须为 1	TR/GR
目的操作数	dst	同 src	(1, c, oh, w)	任意排列但 stride_w 须为 1	TR
			(c, oh_1, oh_0, w)	Free Mode 但 stride_w 须为 1 且 stride_n = (ih_1*ih_0)*stride_h	GR

3.7.8 消息同步指令

通过消息同步机制进行多核间的同步操作。具体指令如下：

```
rvt_dma_msgsend    rs  
rvt_dma_msgwait    rs
```

3.7.8.1 rvt_dma_msgsend

```
rvt_dma_msgsend    rs
```

消息发送指令。rs 包含 message ID 以及等待该消息者的个数。指令的计算逻辑如下所示：

```
msg_id = rs[15:0];  
wait_cnt = rs[31:16];  
msg_que[msg_id].remain_wait_cnt = wait_cnt;  
msg_que[msg_id].sent_cnt++;
```

3.7.8.2 rvt_dma_msgwait

```
rvt_dma_msgwait    rs
```

消息发送指令。rs 包含 message ID 以及等待该消息者的个数。指令的计算逻辑如下所示：

```
msg_id = rs[15:0];  
send_cnt = rs[31:16];  
while(msg_que[msg_id].sent_cnt != send_cnt)  
    continue;  
if(--msg_que[msg_id].remain_wait_cnt == 0)  
    msg_que[msg_id].send_cnt = 0;
```

3.8 同步指令

3.8.1.1 rvt_sync_i

用于保证此指令之前的计算、DMA 指令或所有指令都已执行完。具体指令如下：

```
rvt_sync_i    rs,_engine
```

其中，rs 存放的是 tag，在执行完改指令后会将 tag 写入到 CSR.sync_tag 中去，CPU 可以读取该寄存器来判断指令是否已经执行完，再读取后该寄存器会自动清 0；_engine 表示须同步指令的 engine，0 表示同时同步所有指令（DMA、计算指令等），1 表示同步计算指令，2 表示同步 DMA。

Instruction Listing

fixed 表示固定编码位，resv 表示保留位，— 表示该字段未使用。

Table 1. Configuration / Move / System Instructions

Format	fixed [31:25]	funct5 [24:20]	rs [19:15]	fixed [14:12]	resv [11:7]	fixed [6:0]
cfg.satu rs	0000101	00000	rs	100	—	1011011
cfg.pad rs	0000101	00001	rs	100	—	1011011
cfg.insrt rs	0000101	00010	rs	100	—	1011011
cfg.stencil rs	0000101	00011	rs	100	—	1011011
cfg.round_m ode rs	0000101	00101	rs	100	—	1011011
cfg.rsqrt_ite r rs	0000101	00111	rs	100	—	1011011
cfg.dmaidx rs	0000101	01001	rs	100	—	1011011
msgsend rs	0000101	01010	rs	100	—	1011011
msgwait rs	0000101	01011	rs	100	—	1011011
dma.msgse nd rs	0000101	01100	rs	100	—	1011011
dma.msgwai t rs	0000101	01101	rs	100	—	1011011
sync.i rs, _engine	0000101	01110	rs	100	_engine	1011011
tr ta,rs	0000111	00000	rs	100	ta	1011011
cr ca,rs	0000111	00001	rs	100	ca	1011011
gr ga,rs	0000111	00010	rs	100	ga	1011011
cfg.quant ts	0000111	01001	rs(x0)	100	ts	1011011
cfg.kzp ts	0000111	01010	rs(x0)	100	ts	1011011
smem.bc td,(rs)	0000111	01011	rs	100	td	1011011
smem.dist td,(rs)	0000111	01100	rs	100	td	1011011

Table 2. TIU Instructions

Format	HI [63:32]	fixed [63:57]	resv [56:52]	td [51:47]	fixed [46:44]	ts1 [43:39]	fixed [38:32]	—	—
	LO [31:0]	fixed [31:27]	resv [26]	fixed [25]	ts4 [24:20]	ts3 [19:15]	fixed [14:12]	ts2 [11:7]	fixed [6:0]
cmplt out,a,b,c ,d	HI	0000101	00000	out	011	a	1011011	—	—
	LO	00000	0	1	d	c	011	b	1011011
fcmplt out,a,b,c ,d	HI	0000101	00000	out	011	a	1011011	—	—
	LO	00000	1	1	d	c	011	b	1011011
cmpeq out,a,b,c ,d	HI	0000101	00010	out	011	a	1011011	—	—
	LO	00000	0	1	d	c	011	b	1011011
fcmpeq out,a,b,c ,d	HI	0000101	00010	out	011	a	1011011	—	—
	LO	00000	1	1	d	c	011	b	1011011
cmpgt	HI	0000101	00011	out	011	a	1011011	—	—

Format	HI [63:32]	fixed [63:57]	resv [56:52]	td [51:47]	fixed [46:44]	ts1 [43:39]	fixed [38:32]	—	—
	LO [31:0]	fixed [31:27]	resv [26]	fixed [25]	ts4 [24:20]	ts3 [19:15]	fixed [14:12]	ts2 [11:7]	fixed [6:0]
out,a,b,c,d	LO	00000	0	1	d	c	011	b	1011011
fcmpgt	HI	0000101	00011	out	011	a	1011011	—	—
out,a,b,c,d	LO	00000	1	1	d	c	011	b	1011011
conv	HI	0000101	00100	out	011	x	1011011	—	—
out,x,w,bias,_rq	LO	00000	1	1	0 1 0 _rq -	bias	011	w	1011011
conva	HI	0000101	00100	out	011	x	1011011	—	—
out,x,w,bias	LO	00000	1	1	0 1 1 --	bias	011	w	1011011
fconv	HI	0000101	00100	out	011	x	1011011	—	—
out,x,w,bias,_rq	LO	00000	0	1	0 0 0 _rq -	bias	011	w	1011011
fconva	HI	0000101	00100	out	011	x	1011011	—	—
out,x,w,bias	LO	00000	0	1	0 0 1 --	bias	011	w	1011011
dwconv	HI	0000101	00100	out	011	x	1011011	—	—
out,x,w,bias,_rq	LO	00000	0	1	0 1 0 _rq -	bias	011	w	1011011
fdwconv	HI	0000101	00100	out	011	x	1011011	—	—
out,x,w,bias,_rq	LO	00000	0	1	0 1 1 _rq -	bias	011	w	1011011
pool.avg	HI	0000101	00100	out	011	x	1011011	—	—
out,x,w,_rq	LO	00000	0	1	1 0 0 _rq -	-	011	w	1011011
pool.favg	HI	0000101	00100	out	011	x	1011011	—	—
out,x,w,_rq	LO	00000	0	1	1 0 1 _rq -	-	011	w	1011011
pool.max	HI	0000101	00100	out	011	x	1011011	—	—
out,x	LO	00000	0	1	1 1 0 --	-	011	-	1011011
pool.fmax	HI	0000101	00100	out	011	x	1011011	—	—
out,x	LO	00000	0	1	1 1 1 --	-	011	-	1011011
pool.min	HI	0000101	00100	out	011	x	1011011	—	—
out,x	LO	00000	1	1	0 0 0 --	-	011	-	1011011
pool.fmin	HI	0000101	00100	out	011	x	1011011	—	—
out,x	LO	00000	1	1	0 0 1 --	-	011	-	1011011
fconvdw	HI	0000101	00100	out	011	x	1011011	—	—
out, x, dy	LO	00000	1	1	1 0 0 --	-	011	dy	1011011
fmm.nn	HI	0000101	00111	out	011	x	1011011	—	—
out,x,w,bias,_relu	LO	00000	0	1	1 0 0 - _relu	bias	011	w	1011011
fmm.tn	HI	0000101	00111	out	011	x	1011011	—	—
out,x,w,bias,_relu	LO	00000	0	1	1 0 1 - _relu	bias	011	w	1011011
fmma.n	HI	0000101	00111	out	011	x	1011011	—	—

Format	HI [63:32]	fixed [63:57]	resv [56:52]	td [51:47]	fixed [46:44]	ts1 [43:39]	fixed [38:32]	—	—
	LO [31:0]	fixed [31:27]	resv [26]	fixed [25]	ts4 [24:20]	ts3 [19:15]	fixed [14:12]	ts2 [11:7]	fixed [6:0]
n out,x,w, bias,_rel u	LO	00000	0	1	1 1 0 - _relu	bias	011	w	1011011
fmma.tn out,x,w, bias,_rel u	HI	0000101	00111	out	011	x	1011011	—	—
	LO	00000	0	1	1 1 1 - _relu	bias	011	w	1011011
mm2.nn out,x,w, bias,_rq, _relu	HI	0000101	00101	out	011	x	1011011	—	—
	LO	00000	0	1	0 0 0 _rq _relu	bias	011	w	1011011
mm2.nt out,x,w, bias,_rq, _relu	HI	0000101	00101	out	011	x	1011011	—	—
	LO	00000	0	1	0 0 1 _rq _relu	bias	011	w	1011011
mm2.tt out,x,w, bias,_rq, _relu	HI	0000101	00101	out	011	x	1011011	—	—
	LO	00000	0	1	0 1 0 _rq _relu	bias	011	w	1011011
mm2a.n n out,x,w, bias,_rel u	HI	0000101	00101	out	011	x	1011011	—	—
	LO	00000	0	1	0 1 1 - _relu	bias	011	w	1011011
mm2a.nt out,x,w, bias,_rel u	HI	0000101	00101	out	011	x	1011011	—	—
	LO	00000	0	1	1 0 0 - _relu	bias	011	w	1011011
mm2a.tt out,x,w, bias,_rel u	HI	0000101	00101	out	011	x	1011011	—	—
	LO	00000	0	1	1 0 1 - _relu	bias	011	w	1011011
fmm2.n n out,x,w, bias,_rq, _relu	HI	0000101	00101	out	011	x	1011011	—	—
	LO	00000	0	1	1 1 0 _rq _relu	bias	011	w	1011011
fmm2.nt out,x,w, bias,_rq, _relu	HI	0000101	00101	out	011	x	1011011	—	—
	LO	00000	0	1	1 1 1 _rq _relu	bias	011	w	1011011
fmm2.tt out,x,w, bias,_rq, _relu	HI	0000101	00101	out	011	x	1011011	—	—
	LO	00000	1	1	0 0 0 _rq _relu	bias	011	w	1011011
fmm2a.n n out,x,w, bias,_rq, _relu	HI	0000101	00101	out	011	x	1011011	—	—
	LO	00000	1	1	0 0 1 _rq _relu	bias	011	w	1011011
fmm2a.n	HI	0000101	00101	out	011	x	1011011	—	—

Format	HI [63:32]	fixed [63:57]	resv [56:52]	td [51:47]	fixed [46:44]	ts1 [43:39]	fixed [38:32]	—	—
	LO [31:0]	fixed [31:27]	resv [26]	fixed [25]	ts4 [24:20]	ts3 [19:15]	fixed [14:12]	ts2 [11:7]	fixed [6:0]
t out,x,w, bias,_rq, _relu	LO	00000	1	1	0 1 0 _rq _relu	bias	011	w	1011011
fmm2a.t	HI	0000101	00101	out	011	x	1011011	—	—
t out,x,w, bias,_rq, _relu	LO	00000	1	1	0 1 1 _rq _relu	bias	011	w	1011011
add out,a,b,s hift,_sat u	HI	0000101	01000	out	011	a	1011011	—	—
	LO	00000	0	1	0 0 0 - _satu	shift	011	b	1011011
sub out,a,b,s hift,_sat u	HI	0000101	01000	out	011	a	1011011	—	—
	LO	00000	0	1	0 0 1 - _satu	shift	011	b	1011011
mul out,a,b,s hift,_sat u	HI	0000101	01000	out	011	a	1011011	—	—
	LO	00000	0	1	0 1 0 - _satu	shift	011	b	1011011
mac out,a,b,s hift	HI	0000101	01000	out	011	a	1011011	—	—
	LO	00000	0	1	0 1 1 - -	shift	011	b	1011011
abs out,a	HI	0000101	01000	out	011	a	1011011	—	—
	LO	00000	0	1	1 0 0 - -	-	011	-	1011011
max out,a,b	HI	0000101	01000	out	011	a	1011011	—	—
	LO	00000	0	1	1 0 1 - -	-	011	b	1011011
min out,a,b	HI	0000101	01000	out	011	a	1011011	—	—
	LO	00000	0	1	1 1 0 - -	-	011	b	1011011
sel.gt out,a,b,c	HI	0000101	01000	out	011	a	1011011	—	—
	LO	00000	0	1	1 1 1 - -	c	011	b	1011011
sel.eq out,a,b,c	HI	0000101	01000	out	011	a	1011011	—	—
	LO	00000	1	1	0 0 0 - -	c	011	b	1011011
sel.lt out,a,b,c	HI	0000101	01000	out	011	a	1011011	—	—
	LO	00000	1	1	0 0 1 - -	c	011	b	1011011
lshr out,a,shi ft	HI	0000101	01000	out	011	a	1011011	—	—
	LO	00000	1	1	0 1 0 - -	-	011	shift	1011011
ashr out,a,shi ft	HI	0000101	01000	out	011	a	1011011	—	—
	LO	00000	1	1	0 1 1 - -	-	011	shift	1011011
rshr out,a,shi ft	HI	0000101	01000	out	011	a	1011011	—	—
	LO	00000	1	1	1 0 0 - -	-	011	shift	1011011
fadd out,a,b	HI	0000101	01001	out	011	a	1011011	—	—
	LO	00000	0	1	0 0 0 - -	-	011	b	1011011
fsub out,a,b	HI	0000101	01001	out	011	a	1011011	—	—
	LO	00000	0	1	0 0 1 - -	-	011	b	1011011
fmul out,a,b	HI	0000101	01001	out	011	a	1011011	—	—
	LO	00000	0	1	0 1 0 - -	-	011	b	1011011
fmac	HI	0000101	01001	out	011	a	1011011	—	—

Format	HI [63:32]	fixed [63:57]	resv [56:52]	td [51:47]	fixed [46:44]	ts1 [43:39]	fixed [38:32]	—	—
	LO [31:0]	fixed [31:27]	resv [26]	fixed [25]	ts4 [24:20]	ts3 [19:15]	fixed [14:12]	ts2 [11:7]	fixed [6:0]
out,a,b	LO	00000	0	1	0 1 1 --	-	011	b	1011011
fdiv	HI	0000101	01001	out	011	a	1011011	—	—
out,a,b	LO	00000	0	1	1 0 0 --	-	011	b	1011011
fabs	HI	0000101	01001	out	011	a	1011011	—	—
out,a	LO	00000	0	1	1 0 1 --	-	011	-	1011011
fsubabs	HI	0000101	01001	out	011	a	1011011	—	—
out,a,b	LO	00000	0	1	1 1 0 --	-	011	b	1011011
fmax	HI	0000101	01001	out	011	a	1011011	—	—
out,a,b	LO	00000	0	1	1 1 1 --	-	011	b	1011011
fmin	HI	0000101	01001	out	011	a	1011011	—	—
out,a,b	LO	00000	1	1	0 0 0 --	-	011	b	1011011
fsel.gt	HI	0000101	01001	out	011	a	1011011	—	—
out,a,b,c	LO	00000	1	1	0 0 1 --	c	011	b	1011011
fsel.eq	HI	0000101	01001	out	011	a	1011011	—	—
out,a,b,c	LO	00000	1	1	0 1 0 --	c	011	b	1011011
fsel.lt	HI	0000101	01001	out	011	a	1011011	—	—
out,a,b,c	LO	00000	1	1	0 1 1 --	c	011	b	1011011
cvt.i2i	HI	0000101	01010	out	011	a	1011011	—	—
out,a	LO	00000	0	1	0 0 0 --	-	011	-	1011011
cvt.i2f	HI	0000101	01010	out	011	a	1011011	—	—
out,a	LO	00000	0	1	0 0 1 --	-	011	-	1011011
cvt.f2i	HI	0000101	01010	out	011	a	1011011	—	—
out,a	LO	00000	0	1	0 1 0 --	-	011	-	1011011
cvt.f2f	HI	0000101	01010	out	011	a	1011011	—	—
out,a	LO	00000	0	1	0 1 1 --	-	011	-	1011011
cp out,a	HI	0000101	01010	out	011	a	1011011	—	—
	LO	00000	0	1	1 1 0 --	-	011	-	1011011
not	HI	0000101	01010	out	011	a	1011011	—	—
out,a	LO	00000	0	1	1 1 1 --	-	011	-	1011011
and	HI	0000101	01010	out	011	a	1011011	—	—
out,a,b	LO	00000	1	1	0 0 0 --	-	011	b	1011011
xor	HI	0000101	01010	out	011	a	1011011	—	—
out,a,b	LO	00000	1	1	0 0 1 --	-	011	b	1011011
or	HI	0000101	01010	out	011	a	1011011	—	—
out,a,b	LO	00000	1	1	0 1 0 --	-	011	b	1011011
fmuladd	HI	0000101	01100	out	011	a	1011011	—	—
out,a,b,c	LO	00000	0	1	0 0 0 --	c	011	b	1011011
fsqradd	HI	0000101	01100	out	011	a	1011011	—	—
out,a,b	LO	00000	0	1	0 0 1 --	-	011	b	1011011
fsqrsb	HI	0000101	01100	out	011	a	1011011	—	—
out,a,b	LO	00000	0	1	0 1 0 --	-	011	b	1011011
dq0	HI	0000101	01100	out	011	a	1011011	—	—
out,a,sc ale	LO	00000	0	1	0 1 1 --	-	011	scale	1011011
rq0	HI	0000101	01100	out	011	a	1011011	—	—
out,a,sc ale	LO	00000	0	1	1 0 0 --	-	011	scale	1011011
dq1	HI	0000101	01100	out	011	a	1011011	—	—

Format	HI [63:32]	fixed [63:57]	resv [56:52]	td [51:47]	fixed [46:44]	ts1 [43:39]	fixed [38:32]	—	—
	LO [31:0]	fixed [31:27]	resv [26]	fixed [25]	ts4 [24:20]	ts3 [19:15]	fixed [14:12]	ts2 [11:7]	fixed [6:0]
out,a,scale	LO	00000	0	1	1 0 1 - -	-	011	scale	1011011
rq1 out,a,scale	HI	0000101	01100	out	011	a	1011011	—	—
	LO	00000	0	1	1 1 0 - -	-	011	scale	1011011
sfu.norm out,a	HI	0000101	01100	out	011	a	1011011	—	—
	LO	00000	0	1	1 1 1 - -	-	011	-	1011011
sfu.taylor out,a,coeff	HI	0000101	01100	out	011	a	1011011	—	—
	LO	00000	1	1	0 0 0 - -	-	011	coeff	1011011
sfu.rsqrt out,a	HI	0000101	01100	out	011	a	1011011	—	—
	LO	00000	1	1	0 0 1 - -	-	011	-	1011011
gather.p c out,a,idx, cs,_bdlimit	HI	0000101	01101	out	011	a	1011011	—	—
	LO	00000	0	1	0 0 0 - _bdlimit	cs	011	idx	1011011
scatter.p c out,a,idx, _bdlimit	HI	0000101	01101	out	011	a	1011011	—	—
	LO	00000	0	1	0 0 1 - _bdlimit	-	011	idx	1011011
gather out,a,idx, cs,_bdlimit, _hzd	HI	0000101	01101	out	011	a	1011011	—	—
	LO	00000	0	1	1 0 0 _hzd _bdlimit	cs	011	idx	1011011
scatter out,a,idx, _bdlimit, _hzd	HI	0000101	01101	out	011	a	1011011	—	—
	LO	00000	0	1	1 0 1 _hzd _bdlimit	-	011	idx	1011011
hgatter out,a,idx, cs,_bdlimit	HI	0000101	01101	out	011	a	1011011	—	—
	LO	00000	0	1	1 1 0 - _bdlimit	cs	011	idx	1011011
hscatter out,a,idx, _bdlimit	HI	0000101	01101	out	011	a	1011011	—	—
	LO	00000	0	1	1 1 1 - _bdlimit	-	011	idx	1011011
masksel out_cnt, out,a,mask, _hzd	HI	0000101	01101	out	011	a	1011011	—	—
	LO	00000	1	1	0 0 0 _hzd -	out_cnt	011	mask	1011011
nzidx out_cnt, out,a,_h zd	HI	0000101	01101	out	011	a	1011011	—	—
	LO	00000	1	1	0 0 1 _hzd -	out_cnt	011	-	1011011
trans.cw out,a	HI	0000101	01110	out	011	a	1011011	—	—
	LO	00000	0	1	0 0 0 - -	-	011	-	1011011
trans.wc out,a	HI	0000101	01110	out	011	a	1011011	—	—
	LO	00000	0	1	0 0 1 - -	-	011	-	1011011
lane.cp out,a	HI	0000101	01110	out	011	a	1011011	—	—
	LO	00000	0	1	0 1 0 - -	-	011	-	1011011
lane.bc out,a	HI	0000101	01110	out	011	a	1011011	—	—
	LO	00000	0	1	0 1 1 - -	-	011	-	1011011

Table 3. GDMA Instructions

Format	HI [63:32]	fixed [63:57]	resv [56:53]	dst [52:47]	fixed [46:44]	imm5 [43:39]	fixed [38:32]	—	—	—
	LO [31:0]	fixed [31:27]	resv [26]	fixed [25]	resv [24:22]	src2 [21:16]	src1 [15]	fixed [14:12]	src1 [11:7]	fixed [6:0]
dma.ld td,gs	HI	000011 1	0000	td	011	—	101101 1	—	—	—
	LO	00000	0	1	000	—	gs	011	gs	101101 1
dma.ldt td,gs	HI	000011 1	0000	td	011	—	101101 1	—	—	—
	LO	00000	0	1	001	—	gs	011	gs	101101 1
dma.st gd,ts	HI	000011 1	0000	gd	011	—	101101 1	—	—	—
	LO	00000	0	1	010	—	ts	011	ts	101101 1
dma.stt gd,ts	HI	000011 1	0000	gd	011	—	101101 1	—	—	—
	LO	00000	0	1	011	—	ts	011	ts	101101 1
dma.cp dst,src	HI	000011 1	0000	dst	011	—	101101 1	—	—	—
	LO	00000	0	1	100	—	src	011	src	101101 1
dma.nc trans dst,src	HI	000011 1	0000	dst	011	—	101101 1	—	—	—
	LO	00000	0	1	101	—	src	011	src	101101 1
dma.ld bc td,gs	HI	000011 1	0000	td	011	—	101101 1	—	—	—
	LO	00000	1	1	010	—	gs	011	gs	101101 1
dma.sc p gd,gs	HI	000011 1	0000	gd	011	—	101101 1	—	—	—
	LO	00000	1	1	011	—	gs	011	gs	101101 1
dma.sb c td,src	HI	000011 1	0000	td	011	—	101101 1	—	—	—
	LO	00000	1	1	100	—	src	011	src	101101 1
dma.re d gd,src, _rw,_o p	HI	000011 1	0000	gd	011	_op	101101 1	—	—	—
	LO	00000	1	1	101	_rw	src	011	src	101101 1
dma.re verse dst,src, _dim	HI	000011 1	0001	dst	011	_dim	101101 1	—	—	—
	LO	00000	0	1	001	—	src	011	src	101101 1
dma.m asksel dst,src, mask	HI	000011 1	0010	dst	011	—	101101 1	—	—	—
	LO	00000	0	1	000	mask	src	011	src	101101 1
dma.fm asksel	HI	000011 1	0010	dst	011	—	101101 1	—	—	—

Format	HI [63:32]	fixed [63:57]	resv [56:53]	dst [52:47]	fixed [46:44]	imm5 [43:39]	fixed [38:32]	—	—	—
	LO [31:0]	fixed [31:27]	resv [26]	fixed [25]	resv [24:22]	src2 [21:16]	src1 [15]	fixed [14:12]	src1 [11:7]	fixed [6:0]
dst,src, mask	LO	00000	0	1	001	mask	src	011	src	101101 1
dma.nzi dx dst,src	HI	000011 1	0010	dst	011	—	101101 1	—	—	—
	LO	00000	0	1	010	—	src	011	src	101101 1
dma.fn zidx dst,src	HI	000011 1	0010	dst	011	—	101101 1	—	—	—
	LO	00000	0	1	011	—	src	011	src	101101 1
dma.hg ather dst,src, idx	HI	000011 1	0010	dst	011	—	101101 1	—	—	—
	LO	00000	0	1	100	idx	src	011	src	101101 1
dma.hs catter dst,src, idx,_ac cu	HI	000011 1	0010	dst	011	_accu	101101 1	—	—	—
	LO	00000	0	1	101	idx	src	011	src	101101 1
dma.fh scatter dst,src, idx,_ac cu	HI	000011 1	0010	dst	011	_accu	101101 1	—	—	—
	LO	00000	0	1	110	idx	src	011	src	101101 1

版权与版本说明

版权声明

© 2026 RACE 委员会 · 北京开源芯片研究院 · 北京算能科技有限公司。本文档用于 RISC-V AI 扩展指令集开源发布及技术交流；引用、转载或基于本文档形成衍生材料时，请注明来源、版本及联合发布方。

开源协议

本指令集及相关材料以开源方式发布。许可范围、使用约束、贡献流程和版本管理以正式发布版本所附 LICENSE、NOTICE 及代码/文档仓库声明为准。

联系方式

RACE 委员会 · 北京开源芯片研究院 · 北京算能科技有限公司
官网 / 邮箱: riscv-ai@bosc.ac.cn

版本说明

版本: 开源发布版
对象: Tensor 扩展指令集
日期: 2026 年 · 北京